Unidad 2: Gestión Lean Agile de Productos de Software

Filosofía Lean - Método Kanban

Filosofía Lean

La filosofía Lean es un enfoque de gestión que nació en Japón en la década de 1940 con Toyota, cuyo objetivo principal es maximizar el valor generado para el cliente minimizando el consumo de recursos. Parte de la premisa de que cualquier esfuerzo que no crea valor se considera desperdicio (muda) y debe ser eliminado para concentrar las energías en los aspectos esenciales del producto

7 Principios de Lean aplicados al software

- Eliminar los desperdicios → Es el principio más fundamental; todo paso que no contribuya a generar valor para el cliente debe ser identificado y removido.
- Amplificar el aprendizaje → Fomenta una cultura de mejora continua y transparencia, transformando el conocimiento implícito en explícito a través de la retroalimentación constante.
- Tomar decisiones lo más tarde posible → Consiste en diferir compromisos hasta el último momento responsable, cuando se dispone de mayor información para reducir la incertidumbre.
- Entregar lo antes posible → Se busca acotar los ciclos de desarrollo para entregar rápidamente incrementos funcionales de valor al mercado.
- Potenciar al equipo → Implica dar poder a los trabajadores, respetar su conocimiento y fomentar la autoorganización en lugar del mando y control tradicional.
- Crear integridad (o integridad conceptual) → La calidad no se inyecta al final mediante pruebas, sino que debe estar "embebida" en el sistema desde el inicio del proceso de construcción.
- Visualizar todo el conjunto (Ver el todo): Se busca una visión holística para asegurar que los objetivos de las partes estén alineados con los objetivos globales del negocio

Gastos en la Producción Lean

- Producción en exceso
- Defectos
- Stock
- Pasos Extra en el proceso
- Búsqueda de información
- Esperas
- Transporte

Nta: Por lo que entendí todo esto son más que nada desperdicios (gastos al fin y al cabo) porque para la filosofía Lean cualquier actividad que consume recursos pero no genera valor directo al producto, es desperdicio.

Los desperdicios a su vez se pueden clasificar en desperdicio necesario (que tratamos de minimizarlo) y desperdicio puro (que buscamos eliminarlo).

Los 7 Desperdicios Lean

1. ***Trabajo a medias (Inventario)*** → Requerimientos o código no terminado que "acumula polvo", oculta problemas de calidad y mantiene el dinero inmovilizado sin generar valor.
    - Trabajo parcialmente terminado → servicios basados en conocimiento
2. ***Características extra (Sobreproducción)*** → Desarrollar funcionalidades que el cliente no pidió o no utiliza (basado en que el 80% del valor suele estar en el 20% de las funciones).
    - Características extra → servicios basados en conocimiento
3. ***Procesos extra (Procesamiento excesivo)*** → Pasos innecesarios en el desarrollo, como documentación pesada que nadie lee o aprobaciones redundantes.
    - El mismo nombre → servicios basados en conocimiento
4. ***Cambio de tareas (Movimiento)*** → El multitasking intelectual genera una pérdida de tiempo significativa debido al "tiempo de seteo" o preparación necesaria para cambiar de contexto mental.
    - Cambio de contextos (Seteo) → servicios basados en conocimiento
5. ***Esperas (Demoras)*** → Tiempos muertos esperando por aprobaciones, decisiones o la finalización del trabajo de otros equipos.
    - Esperas/Demoras → servicios basados en conocimiento
6. ***Movimiento (Transporte)*** → En software, se refiere al esfuerzo de transferir conocimiento de un grupo a otro o la distancia física entre los miembros del equipo.
    - Cambio de tareas/Transferencia de conocimiento → servicios basados en conocimiento
7. ***Defectos*** → El retrabajo y la deuda técnica que obligan a rehacer lo que ya debería estar bien, consumiendo capacidad que podría usarse en nuevas funciones.
    - Mismo nombre → servicios basados en conocimiento

A estos se suele sumar un octavo desperdicio: el ***talento no utilizado***, que consiste en ignorar la creatividad y experiencia del personal. Mismo nombre para servicios basados en conocimientos.

Nta: Un proceso basado en conocimientos es aquel que entiende el desarrollo de productos, especialmente el software, no como una línea de montaje repetitiva, sino como un proceso continuo de creación de información y descubrimiento.

Kanban

Kanban es un método diseñado para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento, como el desarrollo de software, actividades creativas o servicios profesionales. Se basa en la visualización de tareas intangibles para asegurar que el sistema opere con la cantidad correcta de trabajo, alineando la demanda del cliente con la capacidad real de entrega del equipo.

Fue desarrollado originalmente por Toyota a finales de la década de 1940 para gestionar la producción industrial mediante el sistema Just-in-Time.

Valores → El método no es solo una herramienta técnica, sino que está guiado por nueve valores fundamentales:

- ***Transparencia*** → Compartir información abiertamente mejora el flujo de valor de negocio. Lenguaje claro y directo
- ***Equilibrio*** → Algunos aspectos (como demanda y capacidad) causarán colapso si no se encuentran equilibrados por un periodo prolongado.
- ***Colaboración*** → El Método Kanban fue formulado para mejorar la manera en que las personas trabajan juntas, por ello, la colaboración está en su corazón.
- ***Foco en el cliente*** → Todo el trabajo del sistema debe orientarse a entregar valor real al cliente o usuario final, y no simplemente a completar tareas internas.
- ***Flujo*** → La realización de ese trabajo es el flujo de valor, tanto si es continuo como puntual. Ver el flujo es un punto de partida esencial en el uso de Kanban.
- ***Liderazgo*** → La habilidad de inspirar a otros a la acción a través del ejemplo, de las palabras y la reflexión. Muchas organizaciones tienen diferentes grados de jerarquía estructural, pero en Kanban, el liderazgo es necesario a todos los niveles para alcanzar la entrega de valor y la mejora.
- ***Entendimiento*** → Conocimiento de sí mismo (tanto individual como de la organización) para ir hacia adelante
- ***Acuerdo*** → El compromiso de avanzar juntos hacia los objetivos, respetando (y donde sea posible, acomodando) las diferencias de opinión o aproximaciones
- ***Respeto*** → Valorando, entendiendo y mostrando consideración por las personas. De manera apropiada al pie de esta lista se encuentra la base sobre la cual reposan el resto de los valores.

Principios

Los principios de Kanban se dividen fundamentalmente en dos grupos: los principios de gestión del cambio y los principios de entrega de servicios. Además de estos seis principios fundacionales, el método se rige por tres principios directores (también llamados "agendas") que impulsan la acción dentro de la organización.

Principios de Gestión del Cambio → Estos principios reconocen que las organizaciones son redes de individuos y buscan reducir la resistencia natural al cambio:

- Empezar con lo que haces ahora: Consiste en entender los procesos actuales tal como se realizan en la realidad y respetar los roles, responsabilidades y cargos vigentes.
- Acordar la búsqueda de la mejora a través del cambio evolutivo: En lugar de imponer soluciones radicales, se busca una evolución gradual y constante.
- Fomentar actos de liderazgo a todos los niveles: El liderazgo no es exclusivo de la alta dirección; se requieren iniciativas desde las contribuciones individuales hasta los niveles más senior.

Principios de Entrega de Servicios → Estos principios ven a la organización como un ecosistema de servicios interdependientes y se centran en el valor entregado:

- Comprender y enfocarse en las necesidades y expectativas del cliente: El foco debe estar siempre en el consumidor del servicio y en el valor que este recibe.
- Gestionar el trabajo y dejar que la gente se autoorganice en torno a él: Se debe poner el foco en el flujo de las tareas y permitir que los trabajadores colaboren para resolverlas.

- Revisar periódicamente la red de servicios y sus políticas: Se busca evolucionar las reglas explícitas para mejorar continuamente los resultados de negocio y hacia el cliente.

Principios Directores (Agendas) → Son llamadas a la acción basadas en las necesidades estratégicas de la empresa:

- Sostenibilidad: Busca encontrar un ritmo de trabajo adecuado, evitando la sobrecarga mediante el equilibrio entre la demanda y la capacidad.
- Orientación al servicio: Se enfoca en que los servicios sean "adecuados para el propósito" (fit for purpose) de los clientes.
- Supervivencia: Se orienta a mantener la competitividad y la adaptabilidad frente a cambios futuros o disruptores del mercado.

Las 6 prácticas de Kanban

Para que un sistema sea considerado Kanban, debe aplicar estas prácticas:

1. ***Visualizar el flujo de trabajo*** → Utilizar tableros y tarjetas para mostrar cómo el trabajo se mueve de una etapa a otra (típicamente de izquierda a derecha y Cada etapa es representada por una columna del tablero, sobre las cuales se aplica la teoría de colas). Construye un modelo visual que refleje cómo se trabaja.

Estas tarjetas permiten indicar cuando el trabajo se puede “pullear” para realizar un trabajo determinado, además de señalar en qué parte del proceso se encuentra.

- Permite absorber y procesar una gran cantidad de información en un corto intervalo de tiempo.
- Habilita la cooperación, ya que todo el mundo tiene la misma imagen.
- Permite tomar decisiones de una manera más rápida y colaborativa.

2. ***Limitar el trabajo en curso (WIP)*** → Establecer máximos numéricos de tareas por columna para forzar el enfoque en terminar tareas antes de empezar nuevas. Limita el trabajo en el sistema a la capacidad disponible, basándose en los datos.

Las políticas para limitar el WiP crean un sistema de arrastre (pull): el trabajo es “arrastrado” al sistema cuando otro de los trabajos es completado y queda capacidad disponible, en lugar de “empujar” estos trabajos al paso siguiente cuando hay nuevo trabajo demandado.

- Cuando los recursos están ocupados completamente, no hay holgura en el sistema y como resultado el flujo es deficiente
- Tener demasiado trabajo no finalizado es un desperdicio de tiempo, de dinero y alarga los tiempos de entrega.

3. ***Gestionar el flujo*** → Monitorear el movimiento del trabajo (flujo) para que sea predecible y fluido, identificando y resolviendo impedimentos rápidamente.

El trabajo fluye sobre un tablero permanente, no existe el concepto de iteración, proceso o proyecto. El tablero puede ser compartido con otros proyectos y otros equipos, ya que se trabaja cada pieza de trabajo de forma individual.

- Limitar el trabajo en curso nos ayuda a asegurar un flujo suave y predictivo.
- El seguimiento y la medición del flujo de trabajo da como resultado información valiosa y útil para gestionar las expectativas de los clientes, hacer predicciones y mejorar.

4. ***Hacer las políticas explícitas*** → Definir reglas claras y visibles para todos sobre cómo se hace el trabajo (ej. Criterios de "Hecho" o priorización).

Las políticas de proceso deben ser escasas, simples, estar bien definidas, visibles, deben aplicarse siempre, y tienen que ser fácilmente modificables. Es una buena práctica poder cambiar fácilmente las políticas, ya que si producen efectos contraproducentes para nuestro proceso o también se considera que no pueden aplicarse las mismas, deben poder cambiarse.

5. ***Establecer ciclos de retroalimentación (Cadencias)*** → Reuniones regulares para revisar el progreso y coordinar el servicio.

Se debe definir con qué frecuencia se realizan las reuniones, donde depende en qué contexto se presentan ya que es importante para el resultado.

Nta: Muy frecuentes → no se ven cambios, No muy frecuentes → bajo rendimiento/hay demasiados cambios que controlar

6. Mejorar colaborativamente y evolucionar experimentalmente → Usar el método científico para probar cambios y mejorar el proceso de forma incremental.

Kanban propone una cultura de mejora continua, donde no introduce cambios significativos en los procesos de desarrollo, sino que identifica ese proceso, lo visualiza (hacer explícito para todos los miembros) y propone mejoras sobre él, las cuales se van aplicando de forma gradual a lo largo del tiempo.

¿Cómo aplicar Kanban?

1. Empezar con lo que se tiene: entender el proceso de desarrollo actual que se está utilizando.

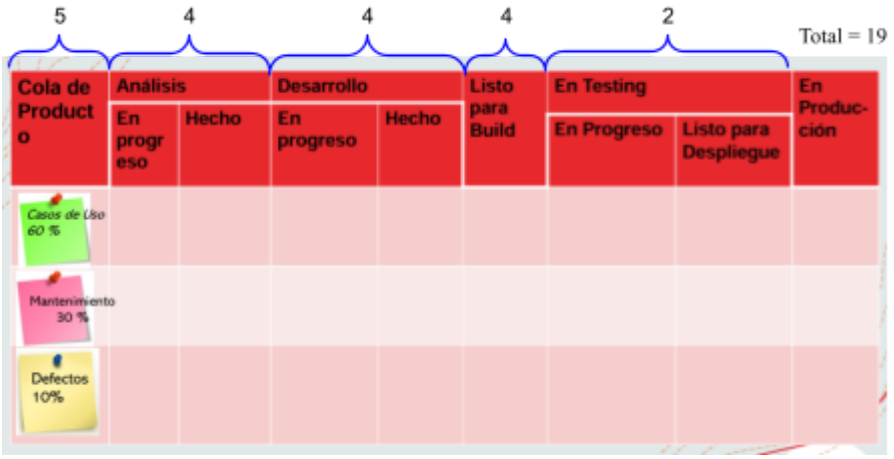
2. Identificar las unidades de trabajo a utilizar (US, CU, defectos).

3. Identificar clases de servicio: diferentes trabajos tienen distintas políticas para tratarlos (DoD). Por ejemplo: requerimientos, defectos, desarrollo y solicitudes.

4. Visualizar el flujo de trabajo diseñando un tablero representando el flujo de trabajo a través de columnas.

5. Definir políticas para cada clase de servicio identificada. Esto implica acordar los WiP para cada columna, asignar color a las tarjetas kan-ban, capacidad de trabajo destinada, fechas de entrega, etc.

6. Identificar cuellos de botella y resolverlos (asignando más recursos o ajustando el WiP donde corresponda).



Nta: Ejemplo de un tablero kanban y los límites WIP. Cada columna es una etapa en el flujo de trabajo y los números son la cantidad de tareas (lím máx) que se puede hacer en cada etapa.

Políticas de Calidad

- ***Definición de Hecho (Definition of Done - DoD)*** → Es una de las políticas de calidad más críticas. Define con precisión cuándo una actividad está terminada y cumple con los estándares necesarios para que el elemento de trabajo pueda ser traccionado (pulleado) al siguiente estado del flujo.
- ***Criterios de Tracción (Pull Criteria)*** → Establecen las condiciones que debe cumplir un ítem para avanzar. Por ejemplo, una política de calidad estricta podría dictar que las historias de usuario no se muevan a pruebas de aceptación hasta que se resuelvan todos los defectos detectados previamente.
- ***Límites de Trabajo en Curso (WIP Limits)*** → Se consideran un tipo de política que ayuda a mantener el enfoque y la calidad, evitando que el equipo se vea desbordado por el exceso de tareas simultáneas.
- ***Gestión de Clases de Servicio*** → Políticas que determinan cómo se priorizan y tratan diferentes tipos de trabajo según su urgencia o impacto en el negocio.
- ***Reposición de Trabajo*** → Reglas sobre cuándo, cuánta cantidad y quién debe añadir nuevos ítems al tablero.
- ***Políticas de Escalamiento*** → Deben estar claramente definidas y documentadas para que todos sepan cómo actuar ante bloqueos o impedimentos graves que afecten la calidad o el flujo

Cadencias

Las cadencias son los eventos o circuitos de retroalimentación (feedback loops) que dirigen la evolución del sistema, permiten realizar cambios y aseguran una prestación de servicio efectiva

1. ***Team Kanban Meeting (Reunión Kanban del equipo):***
    - Frecuencia: Diaria.
    - Propósito: Es una reunión de coordinación donde el equipo se para frente al tablero para observar y seguir el estado y flujo del trabajo (no de los trabajadores). Se enfoca en cómo entregar los elementos más rápido, verificar la capacidad disponible y decidir qué tomar a continuación, además de desbloquear asuntos que impidan el progreso.
2. ***Team Retrospective (Retrospectiva del equipo):***
    - Frecuencia: Quincenal o mensual.
    - Propósito: Es un espacio para reflexionar sobre cómo el equipo está gestionando su trabajo y buscar maneras de mejorar el proceso y la colaboración.
3. ***Team Replenishment Meeting (Reunión de reposición del equipo):***
    - Frecuencia: Semanalmente o a demanda.
    - Propósito: Se utiliza para seleccionar y priorizar los nuevos elementos de la lista de trabajo (backlog) que se introducirán en el sistema para ser realizados a continuación

Métricas

Las métricas más representativas de Kanban son:

- ***Cycle Time*** → Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog. Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.
- ***Lead Time*** → Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entregó.
- ***Touch Time*** → Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), sin tener en cuenta aquellos tiempos de espera (columnas de acumulación).

En cada columna, por ejemplo, en la columna Test de la imagen, se tiene una columna de trabajo y otra de acumulación para poder realizar el concepto de sistema "pull" o de arrastre.

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

- ***Eficiencia del Ciclo de Proceso*** → Esta métrica mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución. Se calcula como Touch time / Lead time.

Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Métricas orientadas al servicio:

- Expectativa de nivel de servicio que los clientes esperan
- Capacidad del nivel de servicio al que el sistema puede entregar.
- Acuerdo de nivel de servicio que es acordado con el cliente.
- Umbral de la adecuación del servicio el nivel por debajo del cual este es inaceptable para el cliente

Métricas en los 3 enfoques

Métrica → Una métrica es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (proceso, producto o proyecto), expresadas de tal forma que permitan una medición objetiva de la realidad.

Todas las métricas poseen un costo asociado, debido a que se deben destinar recursos para su planificación, ejecución y análisis, y no siempre es factible disponer de esos recursos en un proyecto. Esto implica un análisis de costo-beneficio para evitar definir métricas que no aporten un valor a la organización/equipo/proyecto/etc.

Dominio de Métricas

Las métricas se dividen en tres dominios, cada uno con un foco distinto:

- ***Métricas de Procesos*** → Son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software a largo plazo.

Son responsabilidad del Ingeniero de Procesos, quien realiza las mediciones y luego publica los datos para que todos los empleados de la organización puedan acceder a ellos.

Nta: Por lo que entiendo son métricas que se hacen en base a ciertos procesos/proyectos pero no hacen énfasis a ninguno de estos, son datos que buscan centrarse en un proceso en general y no en personas/proyectos específicos

Los indicadores de proceso permiten tener una visión profunda de la eficacia de un proceso, determinando qué funciona de acuerdo a lo esperado y qué no.

Ejemplos:

- ★ Desviación organizacional de estimaciones.
- ★ Defectos por severidad en productos de la organización.
- ★ Errores previos a releases por proyecto.
- ★ Defectos detectados por usuarios.
- ★ Esfuerzo realizado.
- ★ Tiempos de planificación promedio por proyectos.
- ★ Propagación de errores de fase a fase.

- ***Métricas de Proyecto*** → Están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del Líder de Proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.

Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.

Además, se utilizan para evaluar la calidad de los productos en todo momento y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto.

Ejemplos:

- ★ Eficiencia de estimación del proyecto.
- ★ Costos estimados versus costos reales.
- ★ Esfuerzo/Tiempo por tarea del Ingeniero de Software.
- ★ Errores no cubiertos por hora de revisión.
- ★ Fechas de entregas reales versus programadas.
- ★ Cantidad de cambios y sus características.

- Métricas de Producto → Están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto.

Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.

Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para garantizar:

- Que cumplan con los requerimientos del cliente;
- Que cumplan con los requerimientos de calidad;
- Que estén libre de errores;
- Que se realizaron bajo los procedimientos de calidad.

Ejemplos:

- ★ Cantidad de líneas de código del producto;
- ★ Defectos por severidad de un producto;
- ★ Promedio de métodos por clase;
- ★ Cantidad de métodos de cada clase;
- ★ Cantidad de casos de uso por complejidad.

Tradicional

En el enfoque tradicional se hace énfasis en los 3 dominios. Las métricas están basadas en la gestión de proyectos definidos y suele haber una mayor cantidad de métricas en este enfoque que en el resto.

Las métricas constituyen una disciplina transversal, ya que intervienen en todas las etapas del proyecto. Durante la planificación se determina cuáles se utilizarán, cómo se calcularán, quiénes serán responsables de su seguimiento y con qué frecuencia serán analizadas.

No todas las métricas tienen la misma relevancia para todos los participantes del proyecto. Su importancia depende del rol que desempeñe cada persona y de las decisiones que deba tomar. Los perfiles más técnicos suelen enfocarse en métricas relacionadas con el producto y el proceso de desarrollo, mientras que los responsables de la gestión prestan mayor atención a métricas vinculadas al proyecto, como el esfuerzo, los costos y el cumplimiento de los plazos.

Lean (Kanban)

El foco de medición de Kanban es el proceso, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo, es decir, no existe un proyecto con principio y fin. Es decir, se mide el comportamiento del proceso en función al tiempo que se demoran las características.

Las métricas son:

- Lead Time o Elapsed Time
- Cycle Time
- Touch Time
- Eficiencia del ciclo de proceso

Nta: La definición está en el tema anterior

Agile

A diferencia de los enfoques tradicionales, las métricas obtenidas en un equipo o proyecto no suelen ser extrapolables a otros contextos, ya que cada equipo posee características, dinámicas y condiciones de trabajo particulares.

El Manifiesto Ágil destaca la importancia de medir el progreso a través de resultados concretos. Uno de sus principios establece que la principal medida de progreso es el software funcionando. Esto implica que el foco de las métricas debe estar puesto principalmente en el producto entregado y en el valor que este aporta al cliente, más que en la generación de documentación o en el seguimiento exhaustivo de actividades y procesos.

También se promueve la utilización de un conjunto reducido de métricas, seleccionadas en función de su utilidad para el equipo y para el cliente. El objetivo es medir únicamente aquello que aporte información relevante para la toma de decisiones, la mejora continua y la entrega efectiva de valor.

Métrica de velocidad → Es la métrica más importante del enfoque (métrica de producto), y mide la cantidad de Story Points que se realizaron en un Sprint y fueron aceptados por el Product Owner (o sea que se cuentan las US completamente terminadas).

Esta métrica se calcula (no se estima) luego de que el PO. acepte la implementación de una US.

Esta métrica, y sus valores a lo largo de un proyecto ágil, permiten analizar si el equipo posee una estabilidad a lo largo del mismo, lo cual también se relaciona con un principio del manifiesto ágil (desarrollo sostenible).

Métrica de capacidad → Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint.

Se estima al principio de un sprint, por lo que también se la llama velocidad estimada. Se puede estimar en horas ideales o en Story Points.

Running Tested Features (RTF) → Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución.

El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan.

Unidad 3: Gestión de Software como Producto

La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.

Prácticas Continuas

Continuous Integration

Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. Ya que al integrar el código al repositorio de manera frecuente resulta más fácil detectar y corregir los errores.

El objetivo de la integración continua es asegurar que el software pueda ser desplegado en cualquier momento, lo que quiere decir, que debe compilar y ser un producto de calidad en todo momento.

El proceso de validación se refiere a que cada integración ejecuta un conjunto de pruebas automatizadas (como pruebas unitarias en entornos locales, en general con TDD) para detectar y corregir errores tempranamente.

Cada desarrollador en su entorno de trabajo realiza pruebas unitarias y cuando terminó ese componente de código, lo prueba y sabe que funciona, lo sube a un repositorio de integración.

Nta: El tema de TDD está al último, fijate en el índice

Así, la versión del producto está en condiciones de ir a las pruebas de aceptación de usuario sin problemas.

Continuous Delivery

Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento. Suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.

No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción. Es decir que hay un componente «humano» a la hora de tomar la decisión. Pero, en cualquier caso, la versión está lista de inmediato. Esto implica, entre otros, que se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.

La integración continua es prerrequisito de la entrega continua. Con esto se refiere a que los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click. Hay que asegurarse de tener un despliegue automatizado, para que cada puesta en producción no lleve demasiado tiempo, lo que hará que las puestas puedan llegar a hacerse más seguidas, generando que lo que se despliegue no sean demasiados cambios y el riesgo de que algo se rompa disminuya. El software debe estar siempre en un estado de "entregable", es decir, el software buildea, el código se compila y los tests pasan.

Continuous Deployment – Estrategias de Deployments – Canary Deployments – Blue/Green Deployment

Consiste en poner en el ambiente de producción del usuario final el producto. A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.

El propósito de las estrategias es que sea transparente para el usuario que pusiste en producción una nueva versión del producto.

Se recomienda hacer el despliegue a poco tiempo de haber trabajado con los cambios en el código, pues un error en producción un día después de haberlo hecho significa que todavía nos acordamos de lo que hicimos y será más fácil de resolver.

Canary Deployment → Estrategia de lanzamiento gradual o por etapas donde la actualización se despliega primero a un pequeño porcentaje de usuarios para obtener retroalimentación real. Si la prueba es exitosa, se extiende al resto; si falla, el impacto es limitado y la reversión es sencilla.

Esta estrategia nos muestra cómo los usuarios interactúan con los cambios de la aplicación en el mundo real. Al igual que en Blue-Green Deployment, la estrategia Canary ofrece actualizaciones sin tiempo de inactividad y reversiones sencillas. A diferencia de Blue-Green, las implementaciones Canary son más fluidas y las fallas tienen un impacto limitado.

Blue-Green Deployment → Utiliza dos entornos de producción casi idénticos (Azul y Verde). Se transfiere el tráfico de usuarios de la versión anterior (Azul) a la nueva (Verde).

Permite actualizaciones sin tiempo de inactividad y una reversión inmediata volviendo al entorno Azul si ocurre un problema.

Una vez que el tráfico de producción se transfiere por completo de azul a verde, el azul puede quedar en espera en caso de reversión o retirada de producción y se actualiza para convertirse en la plantilla sobre la que se realiza la próxima actualización.

Unidad 4: Aseguramiento de calidad de Proceso y de Producto

Conceptos Generales sobre Calidad

Calidad → Es el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características implícitas esperadas del desarrollo de software profesional.

Está directamente relacionada con la persona (es subjetiva a quién la analice), las circunstancias, el contexto, la edad en un momento de tiempo dado (puede para mí algo no tener calidad, pero para otra persona sí).

Gestión de calidad → La gestión de calidad para los sistemas de software consta de tres puntos de vista fundamentales:

- A nivel de organización, la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad.
- A nivel del proceso, la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados.
- A nivel del proyecto, la gestión de calidad se ocupa también de establecer un plan de calidad para un proyecto.

Aseguramiento de Calidad → Es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad y, en el proceso de fabricación, a la introducción de procesos de calidad.

*Actúa como medida preventiva

El equipo QA debe ser independiente del equipo de desarrollo para que pueda tener una perspectiva objetiva del software.

Planeación de calidad → Es el proceso de desarrollar un plan de calidad para un proyecto. El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valorarán. Por lo tanto, define lo que realmente significa software de “alta calidad” para un sistema particular.

Según Humphrey, un plan de calidad debería seguir la siguiente estructura:

- Introducción del proyecto;
- Planes del producto: fechas de entrega críticas y las responsabilidades para el producto;
- Descripciones de procesos;
- Metas de calidad: Las metas y los planes de calidad para el producto, incluyendo una identificación y justificación de los atributos esenciales de calidad del producto;
- Riesgos y gestión de riesgos.

Problemas en la Calidad

- Atrasos en las entregas → Relacionado a calidad del proyecto.
- Requerimientos no claros → Relacionado a calidad del producto.
- Software que no hace lo que debería → Relacionado a calidad del producto.
- Costos excedidos → Relacionado a calidad del proyecto.
- Trabajo fuera de hora → Relacionado a calidad del proyecto.
- Fenómeno 90-90 (90% hecho 90% restante) → Relacionado a calidad del proyecto.
- No se aplica SCM. → Relacionado a calidad del producto.

Calidad en el desarrollo de Software → El proyecto es el medio que permite alcanzar el producto. Si el medio no tiene calidad, difícilmente se pueda lograr un producto de calidad.

Para que un SW sea de calidad debe satisfacer:

- Las expectativas del cliente;
- Las expectativas del usuario;
- Las necesidades de la gerencia;
- Las necesidades del equipo de desarrollo y mantenimiento;
- Las expectativas de otros interesados.

Principios de Calidad

★ La calidad NO se inyecta, debe estar embebida: la calidad es algo que se concibe desde el momento cero, desde requerimientos en adelante. Lo que se hace con el testing es controlarla.
★ Es una responsabilidad de todos los involucrados en el proyecto.
★ Las personas son la clave para lograrla: se hace mucho hincapié en la capacitación, para brindar herramientas y conocimientos a los involucrados. “Si usted cree que la capacitación es cara entonces pruebe con la ignorancia”. Hacer software es una actividad humano-intensiva
★ Se necesita sponsor a nivel gerencial, pero se puede empezar por uno.
★ Se debe liderar con el ejemplo.
★ No se puede controlar lo que no se mide, debemos usar métricas.
★ Simplicidad, empezar por lo básico.
★ Debe planificarse el aseguramiento de calidad.
★ El aumento de las pruebas no aumenta la calidad. La calidad se obtiene y se inyecta a lo largo del proyecto, no al final.
★ Debe ser razonable para mi negocio.
★ Prevención mejor que corrección: prevenir es menos costoso, ya que corregir demanda un costo muy alto asociado al retrabajo.

Importancia de trabajar con Calidad:

- Es un aspecto competitivo, que permite sobrevivir en el mercado internacional. Las empresas certifican estándares de calidad (de proceso no de producto) para poder competir en el mismo.
- Retiene a los clientes e incrementa los beneficios. Siempre que hablamos de calidad, en el fondo nos referimos al cumplimiento de las expectativas del cliente en todos los sentidos.

- Determina un equilibrio del costo-efectividad. Trabajar con calidad reduce los costos y el retrabajo, ya que se asume que las fallas serán menores.

Visiones de Calidad

A la calidad se la puede analizar desde distintas perspectivas:

Visión del usuario → Esto tiene que ver con las expectativas que tiene el usuario para con el producto, y si este las satisface (subjetivo y difícil de establecer). . El agilismo trata de incluir la visión de calidad del usuario a través del rol de Product Owner, el cual es ocupado por una persona con capacidad de decisión del cliente.

Visión del producto → Esto se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.

Visión del proceso → Si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar, es decir, el proceso aporta valor al producto y no produce desperdicios.

Visión del valor → Encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible y, obviamente generar ganancias con el desarrollo del software.

Visión Trascendental → Esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar, pero son aquellos objetivos que motivan a seguir en busca de la mejora continua.

Calidad en el Software

Los procesos se instancian en los proyectos. Los procesos son sólo una indicación de cómo hacer las cosas, pero es en el proyecto que se insertan actividades para ir regulando si lo que estoy haciendo es lo que se había pensado.

Tenemos las revisiones técnicas que se hacen entre pares y es el producto (cualquier artefacto) el que se somete a la prueba, no quien lo hizo.

Luego tenemos las auditorías que son realizadas por personas externas (los empíricos están muy en contra porque creen que los equipos son capaces de reconocer y corregir por sí mismos sus errores).

*También el testing es otra de las actividades que se pueden ir insertando.

El producto que tengo que someter a evaluación es el que se va generando en el contexto del proyecto, por lo que tengo que insertar tareas para asegurar la calidad del producto.

Los modelos de calidad dicen: hace tu proceso de manera que tenga calidad para vos (empresa), pero este proceso debe ser compatible con lo que yo te digo (modelo de referencia).

Calidad del Producto

Para el caso de la calidad del producto todas las necesidades que se quieren cumplir deberían estar explícitas en los requerimientos, los mismos deben estar completos y bien definidos. Esto quiere decir que para que los requerimientos nos sirvan estos tienen que ser medibles (verificables) y objetivos (no ambiguos).

La gestión de calidad en productos son acciones sistemáticas de las organizaciones para lograr calidad, las cuales requieren equilibrio entre:

- Calidad programada → los alcances del producto planificados.

- Calidad realizada → lo que realmente se ha desarrollado del producto.
- Calidad necesaria → mínimas características que el producto debe tener, para que este satisfaga los requerimientos. En el equilibrio obtenemos las expectativas de calidad que esperamos del producto y todo lo que esté por fuera de esta es desperdicio o insatisfacción

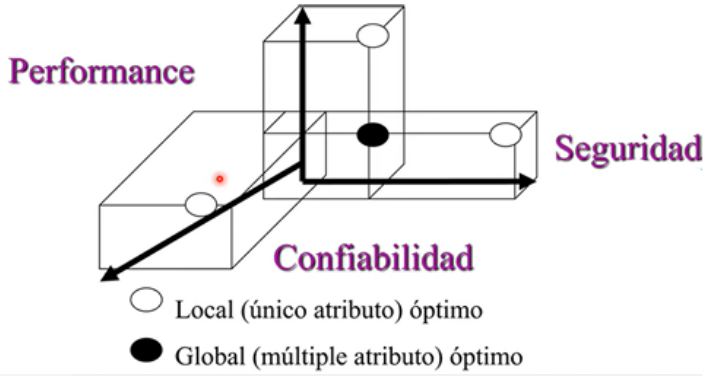
Modelos de calidad de Producto

Al variar los requerimientos para cada producto en particular, no existen modelos de calidad que acrediten para un determinado software qué nivel de calidad posee. Si existen modelos como el Modelo de Barbacci, Calidad de Software (MCCALL) o la ISO 25.010 que permite medir epifenómenos como los RNF del software, para certificar calidad en ciertos aspectos (no los vemos en esta materia).

ISO 25000 -RNF

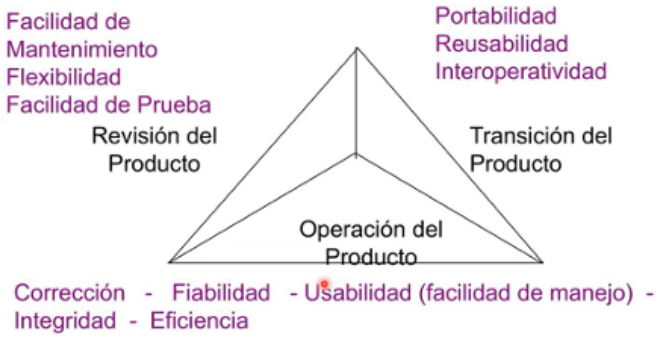


Modelo de Barbacci / SEI



*Busca el equilibrio entre la Perfomance, la Confiabilidad y la Seguridad para evaluar la calidad del producto.

Calidad del SW Mccall



*Tiene tres grandes agrupadores:

- unos relacionados a la capacidad de verificación del producto (revisión del producto)
- otras con el producto en uso (operación del producto)
- otras con los factores que influyen que el producto pueda crecer en el tiempo (transición del producto).

Calidad del Proceso

La calidad del proceso se fundamenta en la premisa de que un proceso con calidad conduce a un producto con calidad. A diferencia de otros factores en el desarrollo de software que son difíciles de manejar, el proceso es considerado el único factor verdaderamente controlable para mejorar el resultado final, ya que el avance tecnológico, las características de las personas y las necesidades del cliente suelen ser ajenos al control total de la organización.

Implementación:

Para garantizar la calidad de un proceso, las fuentes sugieren realizar las siguientes tareas:

- Definir procesos estándares → Establecer cómo deben realizarse actividades críticas como las revisiones técnicas o la gestión de configuración.
- Monitoreo constante → Verificar que el equipo respete los estándares definidos durante el desarrollo.
- No ser rígido → Evitar el uso de prácticas inapropiadas solo porque están en los estándares; el proceso debe adaptarse a la realidad del proyecto.

Objetivos de QA:

El equipo de QA, que debe ser independiente del desarrollo para mantener la objetividad, tiene metas específicas respecto al proceso:

- Controles apropiados → Realizar seguimientos sobre el software y su ciclo de fabricación.
- Cumplimiento de estándares → Asegurar que se respeten los procedimientos y estándares establecidos por la organización.
- Informar desviaciones → Notificar a la gerencia sobre defectos en el proceso o los estándares para que puedan ser solucionados

Estándares de Gestión de Calidad de Software

Estándares de producto → Definen las características que todos los componentes del producto deberían exhibir. Por ejemplo: estilos/buenas prácticas de programación, estándares de documentación.

Estándares de Proceso → Establecen los procesos que deben seguirse durante el desarrollo, por ejemplo: incluyen definiciones de requerimientos, diseño, validación, etc. Definen cómo deben ser implementados los procesos de software.

Definición de procesos

Lo primero que se debe realizar en una organización para lograr tener un proceso de calidad, es definirlo de forma explícita y comunicarlo a todo el equipo, para que así todos tengan una base y el conocimiento de la forma de trabajar. Se deben definir aspectos como etapas, subetapas, roles, qué se debe hacer, en qué momentos se deben hacer, cómo lo deben hacer, etc.

También se deben incorporar disciplinas transversales a ellas, como la Planificación y Seguimiento de Proyectos, SCM y QA, independientemente de la metodología adoptada (tradicional o empírica).

Procesos Definidos → se considera que el proceso es el único factor controlable, por lo tanto, se asume que la mejora de la calidad continua se realiza sobre el proceso. Si el proceso cuenta con calidad, entonces el producto será de calidad.

*Todos los modelos de calidad están basados en procesos definidos

Procesos Empíricos → están basados en la experiencia, por lo que no consideran que la calidad en el proceso determine la calidad en el producto. Por otra parte, no están de acuerdo con las auditorias, ya que asumen que los equipos son autoorganizados. Sin embargo, la calidad en estos procesos se lleva a cabo mediante revisiones técnicas y la constante inspección y adaptación del trabajo.

Actividades relacionadas con el Aseguramiento de la Calidad del Software

Administración de la Calidad de Software

Busca asegurar que se alcancen los niveles requeridos de calidad para el producto de SW, definiendo estándares y proceso de calidad apropiados (que deben ser respetados por todos). El fin en sí de la Administración de Calidad de SW es generar una cultura de calidad y que esta sea vista como una responsabilidad de todos en el equipo.

La idea es insertar acciones (en las distintas actividades del proceso) tendientes a detectar lo más temprano posible oportunidades de mejora sobre el producto y sobre el proceso. Hay que tener ciertas consideraciones respecto al Reporte del Grupo de Aseguramiento de Calidad (GAC):

- No debería reportar la gente que hace calidad al mismo gerente que reporta los proyectos (porque le quita independencia y libertad).
- El grupo de aseguramiento de calidad debería depender del gerente general.
- Cuando sea posible, el grupo de aseguramiento de calidad debería reportar a alguien realmente interesado en el aseguramiento de calidad.

Actividades (algunas):

- ***Aseguramiento de calidad*** → define estándares, procesos, procedimientos y modelos de calidad sobre los cuáles se van a realizar las comparaciones.
- ***Planificación de Calidad*** → se debe planificar la calidad, qué actividades se van a llevar a cabo, cuándo. Se selecciona los estándares aplicables para nuestro proyecto en particular y se los modifica si fuera necesario. Esta actividad abarca determinar los productos de SW que se desea que tengan calidad y determinar sus atributos de calidad más significativos.
- ***Control de Calidad*** → es la ejecución de lo planificado, y se revisa en qué situación está el proyecto. Asegura que los procedimientos y estándares son respetados por el equipo de desarrollo de SW. Existen dos enfoques para el control de calidad:

    - Revisiones de calidad → principal método de validación de la calidad de un proceso o un producto. El grupo examina parte de un proceso/producto + documentación, para encontrar potenciales problemas.
        - Tipos de revisiones de calidad:
            - Inspecciones para remoción de defectos (producto);
            - Revisiones para evaluación de progreso (producto y proceso);
            - Revisiones de Calidad (producto y estándares).
    - Evaluaciones de SW automáticas y mediciones.

Funciones del Aseguramiento de Calidad:

- Prácticas de aseguramiento de calidad.
- Evaluación de la planificación del proyecto de SW.
- Evaluación de requerimientos.
- Evaluación del proceso de diseño.
- Evaluación de las prácticas de programación
- Evaluación del proceso de integración y prueba del software
- Evaluación de los procesos de planificación y control de proyectos.
- Adaptación de los procedimientos de calidad para cada proyecto.

Modelos de Calidad

Modelos para la Mejora de Procesos

La mejora continua de procesos significa comprender los procesos existentes y cambiarlos para incrementar la calidad del producto o reducir los costos y el tiempo de desarrollo.

El propósito de los modelos de mejora es analizar el proceso que tiene la organización y armar un proyecto cuyo resultado, en lugar de ser un producto, es un proceso mejorado que se vuelca de nuevo a la organización con la idea de que se produzca un producto mejor.

Modelo IDEAL (Initiating, Diagnosis, Establishing, Acting, Learning)

Nos da el contexto para crear un proyecto cuyo resultado va a ser un proceso definido. Es un modelo cíclico que mejora el proceso existente en una organización.

Nta: Lo primero que se hace es buscar quien ponga la plata para este tipo de procesos, porque siempre hay algo más importante que ver cómo mejorar algo que ya funciona. (análisis de brecha = donde estamos y adónde queremos llegar)

Lo componen 5 fases:

- ***Inicialización*** → se reconocen las necesidades de cambio en la organización, razones para iniciar, determinar metas buscadas al proponer un cambio en el proceso.
- ***Diagnóstico*** → Establecer la madurez actual de la organización y los riesgos asociados al proceso de mejora (revisar estado actual y futuro de la organización)
- ***Establecimiento*** → Durante la fase se elabora un plan detallado con acciones específicas, entregables y responsabilidades para el programa de mejora basado en los resultados del diagnóstico y en los objetivos que se quieren alcanzar. Para elaborar el plan se parte de definir las prioridades para el esfuerzo de mejora

- ***Ejecutar/Acción*** → Efectuar los cambios y reunir información para aprender de la mejora. Se implementan las acciones planeadas en un proyecto piloto (no en todos los procesos de la organización, ya que es una prueba).
- ***Aprendizaje*** → Busca garantizar que el próximo ciclo sea más efectivo. Durante la misma se revisa toda la información recolectada en los pasos anteriores y se evalúan los logros y objetivos alcanzados para lograr implementar el cambio de manera más efectiva y eficiente en el futuro. Explorar la mejora al resto de proyectos en caso de que sea exitosa la ejecución o realizar correcciones en caso de que fracase el proyecto piloto.

Modelo SPICE (Software Process Improvement Capability Evaluation)

Es un modelo creado para la mejora de procesos de software (ISO 15504). Es una adaptación para la evaluación de procesos de desarrollo de software por niveles de madurez, dividido en 6 niveles.

Es un modelo teórico que consta de dos ***partes***:

- Modelo de Calidad
- SPICE + IDEAL → son ambos modelos de mejora de proceso. Estos modelos de mejora se usan para crear Proyectos de Mejora para una organización. El resultado de este proyecto va a ser un proceso definido que se usará para hacer Proyectos.

*Este modelo establece conjuntos predefinidos de procesos con objeto de definir un camino de mejora para una organización

Niveles de organización:

- Nivel 0 - Organización inmadura → No tiene una implementación efectiva de los procesos
- Nivel 1 - Organización básica → Implementa y alcanza los objetivos de los procesos
- Nivel 2 - Organización gestionada → Gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
- Nivel 3 - Organización establecida → Utiliza procesos adaptados basados en estándares
- Nivel 4 - Organización predecible → Gestiona cuantitativamente los procesos
- Nivel 5 - Organización optimizada → Mejora continuamente los procesos para cumplir los objetivos de negocio

Modelo de Calidad para Evaluar Procesos

Plantean una definición teórica (lineamientos) del proceso que se utilizará en una organización (con diferentes niveles de detalle según lo que se necesite), para luego compararlo con la ejecución del proyecto donde se implementa (instancia) ese proceso de desarrollo. Esto permite medir qué tanto se está cumpliendo en el proyecto con lo que plantea la definición teórica del proceso de desarrollo (definición de roles, asignación de responsabilidades, actividades a realizar, etc.), a través de las auditorías de proyecto.

El objetivo de estos modelos de calidad es acreditar que una organización tiene cierto nivel de madurez en su proceso de desarrollo adoptado, o bien que cierta área de esa organización tiene determinado nivel de madurez.

Capability Maturity Model Integration – CMMI

El modelo CMMI (Capability Maturity Model Integration) es un modelo de referencia descriptivo que proporciona un conjunto de buenas prácticas para la mejora y evaluación de procesos en el desarrollo, mantenimiento y operación de software. Su objetivo principal es guiar a las organizaciones para que alcancen niveles superiores de madurez y predictibilidad en sus resultados.

Constelaciones (Modelos de Negocio)

El CMMI se divide en tres áreas de enfoque según las necesidades de la organización:

- CMMI-DEV (Desarrollo): Guía para medir y administrar procesos de desarrollo de productos y servicios de software.
- CMMI-ACQ (Adquisición): Enfocado en la gestión de la cadena de suministro y la adquisición de productos a terceros.
- CMMI-SVC (Servicios): Orientado a la entrega de servicios internos o externos.

Representaciones del Modelo

Existen dos formas de aplicar y medir el modelo:

- Por Etapas (Staged): Evalúa la madurez organizacional como un todo. Clasifica a las organizaciones en 5 niveles de madurez (del 1 al 5), donde cada nivel es acumulativo respecto a los anteriores.
- Continua (Continuous): Evalúa la capacidad de un proceso individual. Permite a la organización elegir áreas específicas para mejorar y acredita niveles de capacidad (del 0 al 5) para cada una de ellas independientemente.

Niveles de Madurez (Representación por Etapas)

1. Inicial: Procesos inmaduros, improvisados y con resultados impredecibles.
2. Administrado: Los proyectos se planifican, miden y controlan; hay seguimiento de requerimientos y calidad.
3. Definido: Procesos estandarizados en toda la organización, con formación técnica detallada y revisiones entre pares.
4. Cuantitativamente Administrado: Uso sistemático de métricas estadísticas para la toma de decisiones y gestión de riesgos.
5. Optimizado: Foco en la mejora continua e innovación tecnológica de los procesos.

Áreas de Proceso

El modelo cuenta con 22 áreas de proceso en total (16 son comunes a todas las constelaciones). En el Nivel 2 (Administrado), las áreas clave incluyen:

- Gestión de Requerimientos (REQM).
- Planificación de Proyectos (PP).
- Monitoreo y Control de Proyectos (PMC).
- Aseguramiento de Calidad de Proceso y Producto (PPQA).
- Gestión de Configuración (SCM).
- Medición y Análisis (MA).

Características Clave

- Descriptivo, no prescriptivo: El modelo indica qué objetivos deben alcanzarse, pero la organización decide qué prácticas específicas implementar para lograrlos.
- Basado en el éxito histórico: Recopila prácticas de organizaciones que han logrado desarrollos exitosos a lo largo de los años.

- Acreditación SCAMPI: La evaluación para certificar los niveles se realiza mediante un método formal que combina objetividad (evidencias) y subjetividad (juicio del evaluador).

CMMI relacionado con lo Ágil

El proceso de integración y concesiones:

Para que una organización ágil acredite CMMI, ambas filosofías deben adaptarse:

- Lo que Agile debe ceder: Debe aceptar la realización de auditorías externas (monitoreo objetivo) y mantener una trazabilidad formal de los requerimientos, algo que CMMI exige pero que en Agile suele ser transitorio (en el Product Backlog) o estar "persistido" directamente en el código y las pruebas.
- Lo que CMMI debe ceder: El modelo debe flexibilizarse para aceptar registros menos burocráticos; por ejemplo, puede aceptar que no haya una minuta formal de cada reunión diaria (Daily) siempre que se cumplan los objetivos del área de proceso.

Compatibilidad por Niveles

- Nivel 2 (Administrado): Es el punto de mayor encuentro. Las 7 áreas de proceso de este nivel (Gestión de Requerimientos, Planificación, Monitoreo, etc.) pueden recibir soporte de prácticas ágiles si se ejecutan con disciplina.
- Niveles Superiores (3, 4 y 5): La relación se vuelve más desigual y difícil de conciliar. CMMI exige procesos estandarizados en toda la organización y mediciones estadísticas basadas en la premisa de que la experiencia es extrapolable entre proyectos. Por el contrario, Agile sostiene que el conocimiento es situacional y que lo que funciona para un equipo no necesariamente funcionará para otro.

Similitudes fundamentales

A pesar de sus diferencias de forma, ambas comparten objetivos estratégicos:

- Buscan crear organizaciones de alto desempeño.
- Reconocen la importancia vital de la planificación (aunque difieran en su frecuencia y detalle).
- Ambas poseen reglas y políticas internas cuyo incumplimiento tiene repercusiones en la calidad.
- Ninguna es una "bala de plata" aplicable por igual a cualquier tipo de proyecto.

Contrastes de Mentalidad (CMMI vs. Ágil)

Las fuentes destacan diferencias marcadas en sus pilares esenciales:

- Foco: CMMI se centra en que "mejores procesos llevan a mejores productos", mientras que Agile se centra en el software funcionando como principal métrica de progreso.
- Personas: CMMI valora la disciplina, el seguimiento de reglas y la aversión al riesgo. Agile valora la creatividad, la toma de riesgos y individuos capaces de autoorganizarse.
- Mejora: CMMI busca la uniformidad organizacional y el éxito mediante la predictibilidad. Agile busca la innovación en el proyecto y el éxito aprovechando las oportunidades que surgen de la inspección constante.
- Gestión de Riesgos: CMMI tiende a ser proactivo (planes de mitigación previos), mientras que Agile suele ser más reactivo, gestionando el riesgo a través de iteraciones cortas que permiten fallar rápido y barato.

Auditorías de Software

Es una actividad incluida dentro de las disciplinas de soporte, la cual implica una evaluación independiente (realizada por un grupo de trabajo que no tienen nada que ver con el equipo del proyecto) de productos o procesos de software que permiten asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos basada en un criterio objetivo incluyendo documentación.

Beneficios:

- Se obtienen opiniones objetivas e independientes
- Permiten identificar áreas de insatisfacción potenciales del cliente
- Permite asegurar que se están cumpliendo con las expectativas
- Permite dar visibilidad sobre los procesos de trabajo
    - Permite visualizar los puntos fuertes de una organización
    - Permite identificar oportunidades de mejora
- Da visibilidad a la gerencia sobre los procesos de trabajo
- Determinan que se implementen de manera efectiva el proceso de desarrollo organizacional y del proyecto, y las actividades de soporte

Tipos de Auditorías

Auditoría de Proyecto

Es responsable de ver que el proyecto se haya ejecutado con el proceso que se definió. Esto bajo la premisa de que en un proyecto se debe realizar lo que define el proceso, obviamente adaptado a lo que sirve en ese contexto del proyecto particular (teniendo en cuenta que, para obtener una acreditación de calidad, se tiene que ajustar hasta cierto punto al modelo teórico de referencia).

Lo que se hace es tomar evidencias de lo que se está haciendo y contrastarlo con lo documentado en las ERS, arquitectura, diseño, etc.

Acá puede haber diferencias respecto a metodologías tradicionales o empíricas. En SCRUM, por ejemplo, el mismo equipo realiza estas auditorías en la ceremonia de Sprint Retrospective. En cambio, en metodologías tradicionales, el auditor debe ser externo al proyecto y a veces a la empresa.

Auditorías de Configuración Funcional

Valida que el producto cumpla con los requerimientos. La auditoría funcional compara el software que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la ERS.

El propósito de la auditoría funcional es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descritos en la ERS.

Auditoría de Configuración Física

Valida que el ítem de configuración cumpla con la documentación técnica que lo describe (esto permite la trazabilidad y la satisfacción de los requerimientos). La auditoría física compara el código con la documentación de soporte.

Su propósito es asegurar que la documentación que se entregará es consistente y describe correctamente al código desarrollado.

Auditorías Externas de Aseguramiento de Calidad

Son auditorías realizadas por un personal externo a la organización, a quienes se encargan de realizar las auditorías de calidad. Esto permite evaluar a los miembros de QA, para determinar si sus evaluaciones dentro de la organización se están realizando de forma correcta.

Proceso de Auditoría

Roles en una Auditoría

- Auditado → Participa en la auditoría, y es quien propone la fecha de la auditoría, entrega evidencia, contesta las dudas del auditor, propone planes de acción para las desviaciones encontradas, contesta el reporte de auditoría, propone el plan de acción para las deficiencias citadas en el reporte.
- Auditor → puede ser una persona o dos y deben ser de afuera del proyecto que se está auditando. Puede ser el grupo de aseguramiento de calidad, que es el grupo que le da soporte y auditorías de estos tipos.
    - Acuerda la fecha de la auditoría.
    - Comunica el alcance de esta.
    - Recolecta y analiza la evidencia objetiva que es relevante y suficiente para tomar conclusiones acerca del proyecto auditado.
    - Realiza la auditoría.
    - Prepara el reporte.
    - Realiza el seguimiento de los planes de acción acordados con el auditado.
- Gerente de SQA (Software Quality Assurance) → Es quien prepara el plan de auditoría, calcula su costo, asigna recursos, resuelve las no-conformidades entre auditor y auditado. (orientado a la gestión de la auditoría)

Etapas de una Auditoría

1. Preparación y Planificación → El auditado solicita la auditoria y junto con el auditor definen la fecha, el alcance y objetivo, acordando los participantes de esta. Se deben definir métricas de auditoría, como por ejemplo: esfuerzo por auditoría, cantidad de desviaciones, duración de la auditoría, cantidad de desviaciones clasificadas por auditor de CMMI, etc.
2. Ejecución → El auditor escucha lo que la gente dice que hace y luego ve la documentación, pide evidencia de lo que se debería estar haciendo.
    - Reunión de apertura → Se informa cómo será el proceso, indicando lugar y hora
    - Ejecución → se responden las preguntas, se muestra la evidencia y se completa el checklist. El auditor escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose)
3. Análisis y Reporte de Resultados → El auditor realiza el reporte de resultados y el auditado analiza la respuesta, puede o no estar de acuerdo con algunas prácticas y se deja asentado el documento final. Se evalúan los resultados, se hace una reunión de cierre y se hace entrega del reporte final.
4. Seguimiento → Se analizan las desviaciones y prepara un plan de acción que se envía el auditor para que lo revise y apruebe. En función del acuerdo entre auditado y

auditor, puede que el auditor haga un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas.

Herramientas y Técnicas utilizadas

- Checklists → tienen preguntas tipo para garantizar que independientemente de quién haga la auditoría el foco de las cosas que se controlan sea el mismo. Son de mínima, es decir, se debe garantizar de mínimo contestar estas preguntas, pero el auditor puede solicitar más cosas (hacer más preguntas).
- Muestreo → consiste en seleccionar una muestra representativa de los productos y/o procesos a auditar.
- Revisión de registros
- Herramientas automatizadas

Resultados/Hallazgos de una Auditoría

- Buenas prácticas → cuando es algo superador a lo definido que tenemos que hacer. Mucho mejor de los esperado. Ejemplo: se armó una muy buena herramienta para gestionar el plan de proyecto para que la gente pueda accederlo y mejorarlo.
- Desviaciones → cualquier cosa que no se hizo o que no se hizo cómo el proceso lo definió. Hay dos grados:
    - No adecuado = lo que realmente está mal
    - Necesita mejora = si lo estas haciendo pero necesita mejorarse
- Observaciones → son cosas que se advierten por el auditor que no llegan a ser desviaciones pero que pueden llegar a ser riesgosas las prácticas y pueden llegar a generar un problema, por eso se destacan a consideración del equipo para que ellos decidan si hacerlo o no. Algo para revisar. Deberían mejorarse, pero no requieren un plan de acción. Ejemplo: técnica de estimación mala.

Calidad de Producto

Verificación y Validación

Es un proceso de ciclo de vida completo que busca asegurar la integridad del software desde los requerimientos hasta la entrega final.

- Validación: Responde a la pregunta "¿Estamos construyendo el producto correcto?". Su objetivo es asegurar que el software resuelva las necesidades reales del cliente y cumpla con sus expectativas.
- Verificación: Responde a "¿Estamos construyendo el producto correctamente?". Se enfoca en comprobar que el software cumple con los objetivos y especificaciones definidos en la documentación técnica y las líneas base.

Falla → Es un error en un producto de trabajo, donde el producto de trabajo es la salida de cualquier actividad correspondiente al ciclo de vida de desarrollo

Las fallas se originan por problemas de comunicación o por limitaciones de memoria.

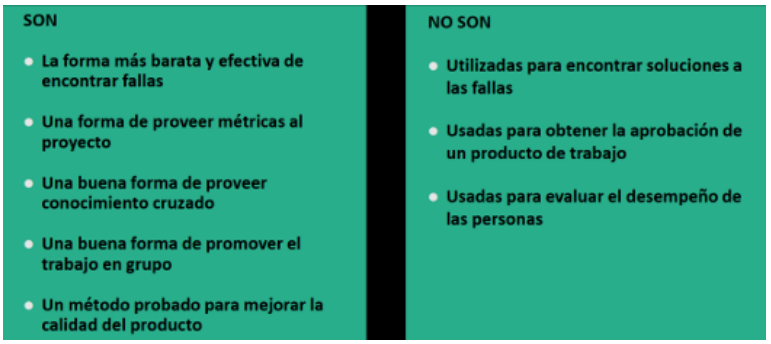


Principios

- Prevenir es mejor que curar
- Evitar es más efectivo que eliminar
- La retroalimentación enseña efectivamente
- Priorizar lo rentable
- Olvidarse de la perfección
- Enseñar a pescar, en lugar de dar el pescado

Revisiones Técnicas - Peer Review

Es un proceso estático de validación y verificación; y no corrige errores.



Objetivo → introducir el concepto de verificación y validación. Busca evitar el retrabajo. Motiva a realizar un mejor trabajo.

Nunca se pone en juicio el autor del artefacto, sólo el artefacto en sí, ya que es necesario que se revelen todos los errores entre los miembros del equipo. Se debe desarrollar una cultura de trabajo que brinde apoyo y no buscar culpables cuando se descubran errores.

Ventajas

- Pueden descubrirse muchos errores
- Pueden inspeccionarse versiones incompletas
- Pueden considerarse otros atributos de calidad

Desventajas

- Es difícil introducir las inspecciones formales
- Sobrecargan al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieran experiencia en su uso
- Requieren tiempo para organizarse y "parecen" ralentizar el proceso de desarrollo

Tipos de revisiones:

- Formales → Tienen un proceso definido con roles
    - Inspecciones

- Informales → Cuando no existe un proceso de cómo realizarlo
    - Walkthrough o recorrido.

Walkthrough o recorrido (informal)

Técnica de análisis estático en la que un diseñador o programador dirige miembros del equipo de desarrollo y otras partes interesadas a través de un producto de software, donde los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas.

Objetivos:

- Mínima Sobrecarga
- Capacitación de Desarrolladores
- Rápido retorno

Esta técnica no obtiene métricas para aprender y dejar registros. Sin embargo, es una de las técnicas más elegidas en los enfoques agiles. Hay una junta de revisión entre colegas después de completar cada iteración del software (una revisión rápida), en la que pueden exponerse los conflictos y problemas de calidad sobre el producto.

Inspecciones (formal)

Tiene un proceso formal y cuenta con un conjunto de roles. Es necesario la utilización de un checklist, que ayuda a la memoria para saber que cosas controlar. Se toman métricas y finalmente se realiza un reporte de la revisión al final de la inspección para analizar los defectos encontrados.

Objetivos:

- Descubrir errores
- Verificar que el software alcanza sus requerimientos
- Garantizar que el software fue construido de acuerdo con ciertos estándares
- Conseguir un software desarrollado de manera uniforme
- Hacer que los proyectos sean más manejables

Roles participantes

- Autor → creador o encargado de mantener el producto a inspeccionar. Inicia el proceso seleccionando a un moderador y junto a este eligen al resto de los roles. Entrega el producto a ser inspeccionado al moderador.
- Moderador → planifica y lidera la revisión. Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector 2 días antes de la reunión. Coordina la reunión de forma tal que no ocurran conductas inapropiadas y realiza un seguimiento de los defectos encontrados.
- Anotador → registra los hallazgos de la inspección. Usualmente termina confeccionado el reporte de la revisión.
- Lector → lee el producto a ser inspeccionado. Este rol es necesario para que los participantes no se dispersen.
- Inspector → Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. todos pueden ser inspectores. Todos pueden inspeccionar.

Nta: Una persona podría asumir varios roles (dependiendo de que roles hablamos)

Etapas del proceso de inspección

1. Planificación → el moderador, a pedido del autor, planifica la inspección definiendo el lugar, el tiempo de duración y los roles. La duración de las reuniones no debe superar las 2 horas, dado que la inspección es un proceso de alto esfuerzo intelectual.
2. Visión general → esta etapa es opcional. El autor realiza una descripción general del producto a inspeccionar.
3. Preparación → es la preparación de cada rol para la reunión. Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos. Esta preparación permite que la reunión de inspección sea más productiva.
4. Reunión de Inspección → el equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos. El lector lee el producto de trabajo y los inspectores comparten los defectos encontrados, los cuales son registrados por el anotador. La reunión finaliza con una conclusión acerca de si se acepta o no el producto de trabajo inspeccionado. Finalmente se realiza un informe detallando que se revisó, por quien, que se descubrió y que se concluyó.
5. Corrección → finalizada la reunión, el autor realiza las correcciones de los defectos encontradas.
6. Seguimiento → Dependiendo de la gravedad puede existir un proceso de re-inspección. En caso de que los defectos sean graves, se realiza nuevamente una inspección. De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige. Otros defectos que no implican una re-inspección, pueden implicar que el autor se reúna con el moderador únicamente para tratar las correcciones realizadas.

Definición de Estándares

Un aspecto importante del aseguramiento de calidad es la definición o selección de estándares que deben aplicarse al proceso de desarrollo de software o al producto de software.

Importancia:

- Se basan en conocimiento sobre la mejor o más adecuada práctica para la organización. Con frecuencia, este conocimiento se adquiere sólo después de una gran cantidad de pruebas y errores.
- Se basan en conocimiento sobre la mejor o más adecuada práctica para la organización. Con frecuencia, este conocimiento se adquiere sólo después de una gran cantidad de pruebas y errores.
- Los estándares aseguran que todos los trabajadores dentro de una organización adopten las mismas prácticas.

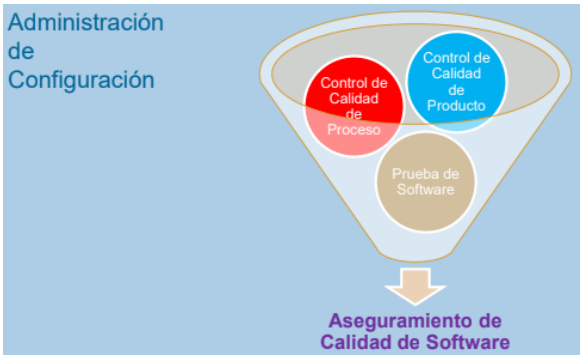
Estándares Para la Gestión de Calidad:

- ***Estándares de producto*** →Se aplican al producto de software a desarrollar. Incluyen estándares de documentos y estándares de codificación.
- ***Estándares de proceso*** → Establecen los procesos que deben seguirse durante el desarrollo, por ejemplo, incluyen las definiciones de requerimientos, procesos de diseño y validación, etc.

Testing

El testing de software, es una de las tareas a realizar en el ámbito del aseguramiento de calidad de software, en conjunto con el control de calidad del proceso y del producto.

Dentro del aseguramiento de la calidad del software, existen actividades destinadas al control de calidad del proceso y del producto. Se realiza un control de calidad sobre el proceso de desarrollo, bajo el argumento de que el proceso de desarrollo posee un gran impacto en la calidad del producto. Es decir, en teoría la calidad del producto depende de la calidad del proceso que se está utilizando.



Luego de desarrollar el software, las actividades de testing revelan la calidad del software en sí, es decir, si el producto satisface con los requerimientos definidos por el cliente en etapas anteriores. Cabe destacar que la actividad de testing no asegura la calidad por sí solo, sino que debe estar acompañada de las demás actividades.

Definición

El testing es una actividad destructiva, por ende su objetivo es encontrar defectos, cuya presencia es asumida de antemano en el código. Y es exitoso cuando efectivamente encuentra defectos en el código.

El Testing de software es un proceso, o una serie de procesos, diseñados para asegurar que el código hace lo que fue diseñado para hacer, o visto de otra manera, que no haga nada que no esté intencionado. El software debería ser predecible y consistente, sin presentar sorpresas a los usuarios.

Testing → Tiene el propósito de encontrar defectos, algo que funcione mal

QA (Quality Assurance) → Tiene un propósito más preventivo, actuando como una medida para evitar que los errores ocurran en primer lugar

Nta: Un sw con alta calidad y una buena implementación de QA va a conllevar a un no exitoso proceso de testing (si está bien hecho no debería tener tanto errores duh)

Nos ahorramos los costos que conllevan la reparación de defectos en una etapa ya tardía (los cuales creería que son caros :))

Nunca se van a poder cubrir el 100% de las pruebas, por el temilla del tiempo y el costo. Por lo que vamos a tener que charlar con el cliente para acordar cuánta cobertura va a tener el sistema.

Principios

- ★ El testing muestra presencia de defecto → Las pruebas pueden demostrar que existen errores, pero no pueden probar que no los hay. El testing asume que existen defectos y los busca
- ★ El testing exhaustivo es imposible → El testing debe centrarse en seleccionar un subconjunto de casos de prueba con la mayor probabilidad de encontrar la mayor cantidad de errores

Nta: Es imposible probar todas las combinaciones posibles de entradas y caminos lógicos, así que pensá cómo hacer para reducir la cantidad de casos de prueba.

- ★ Testing temprano → Las actividades de prueba deben comenzar lo antes posible en el ciclo de vida del desarrollo, incluso antes de que exista código ejecutable, analizando los requerimientos

Nta: Evitar el defecto lo antes posible es más barato de solucionar que si se descubre cuando ya esta el producto listo.

- ★ Agrupamiento de Defectos → Indica que la mayoría de los defectos suelen concentrarse en un conjunto limitado de módulos o áreas del software. Si se encuentran muchos errores en una sección específica, existe una alta probabilidad de que esa misma sección contenga aún más defectos ocultos
- ★ Paradoja del Pesticida → Si se ejecutan las mismas pruebas una y otra vez, eventualmente dejarán de encontrar nuevos defectos, de la misma forma que los insectos desarrollan resistencia a los pesticidas. Para superar esto, los casos de prueba deben ser revisados y actualizados regularmente.
- ★ El testing es dependiente del contexto → El valor y la forma de aplicar las prácticas de prueba dependen totalmente del entorno y tipo de proyecto
- ★ Falacia de la ausencia de errores → Encontrar y corregir muchos defectos no sirve de nada si el sistema resultante es inutilizable o no satisface las necesidades y expectativas reales del usuario
- ★ Un programador debería evitar probar su propio código
- ★ Una unidad de programación no debería probar sus propios desarrollos → Al igual que el individuo, una organización de desarrollo puede carecer de objetividad al evaluar su propio trabajo, ya que el proceso de encontrar errores puede percibirse como una amenaza para cumplir con el calendario y presupuesto.
- ★ Examinar el software para probar que no hace lo que se supone que debería hacer es la mitad de la batalla, la otra mitad es ver que hace lo que no se supone que debería hacer.
- ★ No planificar el esfuerzo de testing sobre la suposición de que no se van a encontrar defectos.

Mitos

- El Testing es una etapa que comienza al terminar de codificar → El testing es un conjunto de actividades que acompañan y retroalimentan al desarrollo desde el principio.
- El Testing es probar que el software funciona → No, por lo anterior.
- TESTING = CALIDAD de PRODUCTO → El testing pone en evidencia la calidad, pero no la agrega ni la garantiza por sí solo. La calidad debe estar "embebida" o construida en el producto desde el inicio del ciclo de vida
- TESTING = CALIDAD de PROCESO → El testing se enfoca en el producto, el QA se enfoca en el proceso. Así que mal mal mal
- El tester es el enemigo del programador → El tester te ayuda a entregar algo de calidad, no te quiere cagar la vida

Conceptos Importantes

Defecto vs Error → La diferencia entre ambos conceptos es el momento en el cual se detectan y solucionan los errores o defectos. Un error es detectado y corregido en una misma etapa, y un defecto es un error que se traslada de una etapa a otra etapa posterior en la cual se introdujo. El testing encuentra defectos, ya que son errores que surgieron en etapas anteriores, pero se detectan en la etapa de prueba.

Defecto

Un defecto posee dos características principales, que permiten catalogarlos en la etapa de testing:

- Severidad → Define la gravedad del defecto, y es determinada por la persona que realiza el testing, por lo que esta característica es de naturaleza técnica. Y sigue la siguiente escala:
    - Bloqueante → El defecto no permite continuar con la ejecución del sistema
    - Crítico → El sistema funciona, pero la funcionalidad que se está testeando tiene un defecto crítico
    - Mayor → La funcionalidad que se está testeando funciona, pero no de forma correcta
    - Menor → La funcionalidad se ejecuta correctamente, pero con advertencias erróneas o errores de baja importancia.
    - Cosmética → Formato de fechas, formato de números, distribución de componentes en una GUI, temas de presentación de interfaces, etc.
- Prioridad → La prioridad define el impacto del defecto en la funcionalidad para el negocio, y permite ordenar la atención de los defectos según las necesidades del cliente. Una escala posible:
    - Urgencia
    - Alta
    - Media
    - Baja

Estas características son independientes entre sí, ya que varían dependiendo del tipo de negocio y de sistema que se está desarrollando.

Casos de Prueba

Son una secuencia de pasos, variables y condiciones que hay que tener en cuenta para obtener un resultado esperado (para ver si el sistema funciona).

Es el artefacto más importante del testing. Este se construye una vez, pero sirve para realizar infinitas pruebas (las cuales todas tienen que tener el mismo resultado, si el resultado difiere entonces es otro caso de prueba??). La característica más importante de los casos de prueba es ser REPRODUCIBLES. Si se encuentra un defecto y no es reproducible a través de un caso de prueba, no es un defecto.

Nta: Es la salida del proceso de testing y se construyen en base a los requerimientos del sistema. Si los requerimientos/CU están bien definidos, entonces es más fácil hacer los casos de pruebas

Los casos de prueba se derivan de diferentes fuentes:

- Documentos del cliente

- Información relevada
- Requerimientos
- Especificaciones de programación
- Código

Lo que se busca es crear la menor cantidad de casos de prueba para la mayor cantidad de defectos encontrados por los mismos. El objetivo de los casos de prueba es descubrir defectos.

El Caso de prueba está compuesto por:

★ Objetivo → la característica del sistema a comprobar
★ Datos de entrada y de ambiente → datos a introducir al sistema que se encuentra en condiciones preestablecidas.
★ Comportamiento esperado → La salida o la acción esperada en el sistema de acuerdo a los requerimientos del mismo.

Ciclos de Prueba

El concepto de ciclos de pruebas se refiere a la ejecución de un conjunto de casos de prueba sobre una versión determinada de un producto. Es una actividad fundamental para validar el estado del software tras el proceso de desarrollo o de corrección de errores.

Tipos de ciclos:

- Ciclo 0 → Es la primera ejecución de las pruebas. Generalmente es de carácter manual y sirve para configurar el entorno, verificar los datos y establecer la línea base de calidad.
- Ciclo 1 y posteriores → A partir de la segunda vuelta, las pruebas suelen automatizarse. Si se detectan defectos en un ciclo, el producto vuelve a desarrollo para su corrección y, al recibir una nueva versión corregida, se inicia un nuevo ciclo

Regresión en los ciclos:

Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados.

Nta: Generalmente arreglar un defecto, te va a generar un error nuevo, por lo que a veces no es tan buena idea la regresión (costo, esfuerzo y tiempo). Además hay que tener en cuenta que vamos a tener que realizar todas las pruebas de nuevo aunque antes no se hayan detectado defectos, porque no podemos controlar/saber exactamente dónde va a impactar un arreglo que hagamos.

Existen dos formas principales de abordar los ciclos sucesivos después del Ciclo 0:

- Ideal (con regresión exhaustiva): Consiste en ejecutar todos los casos de prueba en cada nuevo ciclo para asegurar que las correcciones no hayan roto funcionalidades que antes andaban bien.
- Práctico (sin regresión total inmediata): Por razones de tiempo y costo, muchas veces se ejecutan solo los casos que fallaron previamente. No obstante, se recomienda que, una vez corregidos todos los defectos, se realice un ciclo final de regresión completa para recuperar la confianza en el sistema

Niveles de Prueba

Los niveles de prueba determinan el foco o la granularidad de la prueba que se está por ejecutar. En general, primero se comienzan probando componentes pequeños, y luego se integran en pruebas de mayor granularidad. Esto es al revés de cómo se desarrollan los componentes.

Pruebas Unitarias → Las hace el desarrollador antes de integrar la funcionalidad el código y están incluidas en el DoD. Son pruebas que se realizan sobre un componente, de manera independiente a los otros. Se suelen reparar los errores apenas se encuentran, sin registrarlos formalmente.

Pruebas de Integración → Una vez que las unidades han sido probadas individualmente, este nivel se enfoca en verificar cómo interactúan y funcionan juntas. Tiene el propósito de detectar defectos en las interfaces y la comunicación entre componentes. Su objetivo es verificar que los componentes ensamblados funcionen de forma coherente

Se suele llevar a cabo una prueba de integración incremental, desde lo más general (que abarca más componentes) a lo más específico (top-down) o desde lo más específico a lo más general (bottom-up)

Pruebas de sistema → Este nivel evalúa el producto completo e integrado para determinar si cumple con los requerimientos y objetivos originales. Su propósito es comparar el sistema contra sus objetivos de negocio y documentación de usuario, buscando discrepancias que no fueron detectadas en niveles previos.No se limita a la funcionalidad; incluye pruebas de volumen, estrés, seguridad, rendimiento, configuración y documentación, entre otras.

Debe ser realizado por un grupo independiente de personas (ajenas al desarrollo) para mantener la objetividad y una mentalidad de usuario final.

Pruebas de Aceptación de Usuario → Es la validación final del software frente a las necesidades reales del usuario y los requerimientos del contrato. Su propósito es brindar confianza al cliente de que el sistema está listo para ser desplegado y satisface sus expectativas de negocio.

Tipos principales:

- Pruebas Alfa → Realizadas por el usuario en un entorno de laboratorio o desarrollo.
- Pruebas Beta → Realizadas por usuarios finales en sus propios entornos de trabajo reales antes del lanzamiento oficial.

Testing en Ambientes Ágiles → En metodologías como Scrum o XP, estos niveles no siempre son fases secuenciales largas. Las pruebas unitarias y de integración ocurren continuamente dentro de cada iteración (sprint) de 1 a 4 semanas, y las pruebas de aceptación se presentan al final de cada incremento funcional.

Ambientes

Son todos los recursos, tanto de hardware como software, que se requieren para poder trabajar con el producto. Existen distintos ambientes en el desarrollo de software, los cuales poseen distintas necesidades, según la etapa de desarrollo en la que se encuentre el software.

Desarrollo → Es el entorno utilizado por los programadores para construir el software. Implica el hardware y software necesario (librerías, extensiones, IDEs, compiladores, etc) para poder desarrollar y desplegar el producto y utilizarlo.

Su propósito es permitir la codificación y las pruebas iniciales sin afectar el trabajo de otros.

Aquí se realizan predominantemente las pruebas unitarias y, generalmente, las de integración. En este nivel, los programadores suelen trabajar en sus propios "sandboxes" (areneros) para probar código nuevo antes de integrarlo al repositorio compartido.

Pruebas → Es un ambiente dedicado exclusivamente a las actividades de verificación, al cual los desarrolladores no deben tener acceso para garantizar la objetividad. Su propósito es ejecutar casos de prueba de manera sistemática y metodológica para encontrar defectos reproducibles.

En este ambiente se realizan las pruebas de sistema o de versión.

Preproducción → Este ambiente actúa como un ensayo final y debe ser una réplica lo más fiel posible de la configuración de producción. Su propósito es validar que el producto funcionará correctamente una vez desplegado ante el cliente final y que el proceso de instalación sea exitoso.

Se utiliza para las pruebas de aceptación de usuario (UAT), pruebas de rendimiento, carga y estrés, ya que permite medir el comportamiento del sistema en una infraestructura equivalente a la real.

El problema de este entorno es que muchas veces resulta difícil (o imposible en algunos casos), debido a que es muy costoso replicar principalmente las configuraciones de hardware.

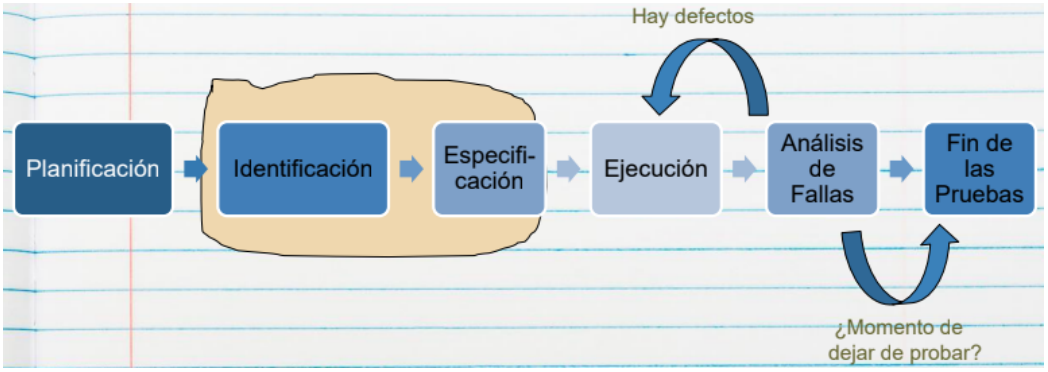
Producción → Es el entorno real donde el software es utilizado por los usuarios finales para sus actividades cotidianas. Su propósito es ejecutar la versión estable del producto que ya cumple con todos los criterios de "Hecho" (Definition of Done).

Obviamente en este entorno no se realizan pruebas, ya debería estar listo para desplegar. Solo en casos donde es imposible replicar la infraestructura (como grandes servidores mainframe), se realizan pruebas de rendimiento en producción, asumiendo riesgos elevados y monitorizando constantemente los recursos para detener la prueba si se superan umbrales críticos.

Proceso de Pruebas

Testing Ad Hoc: se realiza testing sin ningún proceso definido, perdiendo la trazabilidad ya que no se registra el paso a paso de lo que se probó, sino que se prueba libremente sin llevar ningún registro.

El testing es un proceso definido (que se lleva a cabo durante todo el ciclo de vida del producto), por lo que está compuesto de etapas detalladas. Generalmente el proceso de pruebas es estándar (puede tener algunas variaciones):



Planificación → Determina cómo se incluirá el Testing en el plan del proyecto, y cómo será el Test Plan (que recursos se usará, que riesgos se tendrá, cuál será el criterio de aceptación, qué entornos se emularan, quién realizará cada Testing, cuando, etc.).

El resultado de la planificación es el Plan de Pruebas, que debe contener:

- Riesgos y objetivos del testing
- Estrategia de Testing
- Recursos
- Criterio de Aceptación

Diseño (Identificación y especificación de casos de prueba) → Revisando las bases de la planificación del Testing, se identifican los datos necesarios, diseñan y priorizan los CP que se llevaran a cabo (se define que entorno se usara, como se llevarán a cabo, por quien, cuando, etc.). Analiza si los requerimientos son testeables o no. Se define si se usa regresión o no.

Ejecución → Cuando se ejecutan los CP, se registran y se comparan los resultados generando un reporte de defectos (con defectos encontrados, condiciones, entornos). Se trata de automatizar lo más que se pueda respecto de estas ejecuciones (ahorrando tiempo/costo). Incluye: Creación de los datos necesarios para la prueba, automatización de todo lo que sea necesario, implementar y verificar el ambiente, ejecutar los casos de prueba, registrar los resultados de la ejecución y comparar los resultados reales con los esperados.

Análisis de fallas (Evaluación y Reporte) → Se hace un seguimiento de la corrección de los defectos encontrados hasta que se cierren todos los CP, es decir que se hayan solucionado todos. Se evalúa el criterio de aceptación, se reporta el resultado de las pruebas a los interesados, se verifica los entregables y que los defectos se hayan corregido y se evalúa cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.

Acá se confecciona el informe de reportes.

Fin de las pruebas → En las empresas más maduras se deja de probar recién cuando no hay defectos bloqueantes, críticos, mayores y los defectos menores y cosméticos son muy pocos, los que se ponen en la nota de Release. Se confecciona el informe final (opcional).

Estrategias de Prueba

Se utilizan para pasar por la mayor cantidad de funcionalidades con la menor cantidad de casos pruebas confeccionados, debido a que el tiempo y el presupuesto para diseñarlos y ejecutarlos es limitado.

Caja Negra → En esta estrategia no se dispone de la estructura interna de la implementación, sino que se analizan las funcionalidades en términos de entradas y salidas de esa "caja".

El proceso consiste en ingresar determinados datos a esa funcionalidad vista como caja negra, y luego comparar los resultados obtenidos con los resultados esperados. Para ello, se realiza un testing exhaustivo de entrada, probando cada posible entrada conducida como caso de prueba, no necesariamente las válidas. En transacciones no sólo se debe considerar todos los datos posibles, sino todas las secuencias posibles de transacciones previas.

Se puede clasificar en:

- ***Métodos basados en especificaciones*** → se ejecutan utilizando la documentación de especificaciones realizadas del producto.
    - Partición de equivalencias → Consiste en dividir el dominio de entrada de un programa en un número finito de clases de datos en base a una condición externa del sistema. Se asume que probar un valor representativo de cada clase es equivalente a probar cualquier otro valor de la misma categoría
    - Análisis de los valores límites → Se basa en el principio de que los errores suelen ocurrir con mayor frecuencia en los extremos o fronteras de los rangos de entrada en lugar de en el centro.

      Se seleccionan casos de prueba justo en, por encima y por debajo de los bordes de las clases de equivalencia
- ***Métodos basados en experiencia*** → Se determinan las entradas del sistema y se analizan los resultados en base a la experiencia y el conocimiento del tester
    - Adivinanzas de defectos → Como tester que conoce el software y tengo conocimientos sobre testing (no es necesario pero sirve bastante), por intuición creo una lista de posibles defectos o situaciones que pueden dar error. Y luego hago pruebas a partir de esto.
    - Testing exploratorio → el tester mientras va probando el software, va aprendiendo a manejar el sistema y junto con su experiencia y creatividad, genera nuevas pruebas a ejecutar.

Caja Blanca → consiste en diseñar casos de prueba basándose en el conocimiento de la estructura interna o lógica del software, como su código fuente o esquemas de base de datos.

Tiene dos fallas:

- El número de caminos lógicos únicos puede ser sumamente grande, tomando muchísimo tiempo de probar absolutamente todas de ellas.
- Las pruebas de camino exhaustivas no aseguran que el programa sea el correcto para las especificaciones, tampoco detecta caminos faltantes ni determina errores de datos sensibles.

Se pueden utilizar los siguientes métodos en la estrategia:

Cobertura de sentencias → Consiste en escribir suficientes casos de prueba para que cada sentencia del programa se ejecute al menos una vez.

Es una métrica débil. Incluso con un 100% de cobertura de sentencias, pueden quedar errores sin detectar, como fallos en la lógica de las decisiones o caminos que nunca se recorren

Cobertura de decisión →Requiere que cada decisión (como un if, while o switch) tenga un resultado verdadero y uno falso al menos una vez en el transcurso de las pruebas. Generalmente satisface la cobertura de sentencias, ya que al recorrer todas las direcciones de una rama, se suelen ejecutar todas las sentencias asociadas.

Aunque es más fuerte que la anterior, sigue siendo insuficiente para detectar errores complejos dentro de condiciones compuestas

Cobertura de condición → En este método, se diseñan pruebas para asegurar que cada condición individual dentro de una decisión tome todos los resultados posibles (verdadero/falso) al menos una vez.

Cumplir con la cobertura de condición no siempre garantiza cumplir con la de decisión (se pueden alternar condiciones sin cambiar el resultado final de la decisión lógica).

Cobertura de decisión condición → Es un enfoque híbrido que requiere que cada condición individual en una decisión tome todos los resultados posibles Y que la decisión completa también tome todos los resultados posibles al menos una vez

Cobertura múltiple → Este criterio exige que se prueben todas las combinaciones posibles de los resultados de las condiciones dentro de cada decisión.

Es muy exhaustiva para decisiones con lógica compleja, ya que utiliza esencialmente una tabla de verdad para generar los casos de prueba

Cobertura de enunciados o caminos básicos → Este método, propuesto originalmente por Thomas McCabe, utiliza la teoría de grafos para identificar el conjunto de caminos linealmente independientes a través del código.

Complejidad Ciclomática → Se calcula este número (V(G)) para determinar cuántos caminos básicos existen.

Se selecciona un "camino base" (flujo normal) y luego se van "invirtiendo" las decisiones una a una para identificar los demás caminos que conforman la base del programa.

Garantiza que se cubran todos los resultados de las decisiones (cobertura de decisión) y que se evalúe la "esencia" del flujo de control del programa.

Tipos de Prueba

Smoke Test → Es un tipo de prueba que se hace para validar que no haya fallas groseras, de gran magnitud, en el producto de software. Se realiza antes de comenzar el ciclo 0. Nos ahorra empezar a testear formalmente si encuentra un error, porque todavía hay algo que corregir.

Nta: una revisión rápida para ver si el producto está en condiciones de pasar al ciclo de prueba

Testing Funcional → Controla que el software se comporte de la misma manera que lo especificado en la documentación, cumpliendo con las funcionalidades y características definidas.

Se basa en los requerimientos funcionales y el proceso de negocio. Hay testing funcional basado en dos aspectos:

- ***Basado en requerimientos*** → cuando se prueban requerimientos específicos, apunta a probar una funcionalidad sola (utilizan a los requisitos definidos en una ERS o los acuerdos que contienen las pruebas de usuario y los criterios de aceptación de una US para realizar las pruebas.)

- ***Basado en proceso de negocio*** → cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso.

Testing No Funcional → Se basa en cómo trabaja el sistema haciendo foco en los requerimientos no funcionales (foco en el "cómo", no en el "que"). Son las pruebas más complejas, por su gran dependencia al entorno del sistema, por lo que debe ser lo más parecido posible al del cliente.

Nta: hay características que no se pueden probar, como el ancho de banda del internet, la seguridad o performance en una determinada situación

Incluye varios tipos de prueba como:

- ***Performance*** → Se ve el tiempo de respuesta (escenario esperado respecto a los tiempos de respuesta) y la concurrencia. Deben pasar esta prueba sí o sí.
- ***Carga*** → no solo mira performance, mira el comportamiento de los dispositivos de hardware (procesadores, discos, etc.) y de las comunicaciones.
- ***Stress*** → queremos forzar al sistema para que falle, se lo somete a condiciones más allá de las normales. Se ve el tiempo que hay que esperar para probar nuevamente el sistema y la robustez del sistema (tiempo de recuperación).
- ***Mantenimiento*** → para ver si el producto está en condiciones de evolucionar, se controla que haya documentación, manual de configuración, etc. Se observa la facilidad que existe para corregir un defecto.
- ***Usabilidad*** → que sea cómodo para el usuario.
- ***Portabilidad*** → se prueba en los distintos entornos acordados con el cliente.
- ***Fiabilidad*** → probamos que podemos depender del sistema. Resultados que se obtienen, seguridad física del software.
- ***De interfaz de usuario*** → suelen ser más complejas las GUI's que las interfaces de comandos.
- ***De configuración***.

Testing en Agile

El testing ágil es un enfoque colaborativo que involucra a todo el equipo (desarrolladores, testers y clientes) con el fin de entregar software de alta calidad que aporte valor real al negocio. A diferencia de los métodos tradicionales, las pruebas no son una fase final aislada, sino una actividad continua integrada en ciclos de desarrollo cortos e iterativos.

Manifiesto del Testing

El manifiesto del testing ágil establece una serie de valores donde se priorizan ciertos elementos sobre otros para mejorar la calidad del software:

- Pruebas durante el desarrollo sobre pruebas al final.
- Prevenir defectos sobre encontrar defectos.
- Entender lo que se está probando sobre verificar la funcionalidad.
- Construir un mejor sistema sobre romper el sistema.
- Responsabilidad de todo el equipo por la calidad sobre el tester es el único responsable de la calidad.

Los 9 Principios del Testing Ágil

1. El testing se mueve hacia adelante en el proyecto (testing temprano).
2. El testing no es una fase aislada, sino una actividad continua.

3. Todos hacen testing; la calidad es responsabilidad compartida.
4. Reducir la latencia del feedback para corregir errores rápidamente.
5. Las pruebas representan expectativas del cliente y guían el desarrollo.
6. Mantener el código limpio, corregir los defectos rápido
7. Reducir la sobrecarga de documentación de las pruebas
8. Las pruebas son parte del “done”
9. De probar al final a Conducido por Pruebas

Prácticas del Testing Ágil

Las prácticas ágiles integran el testing en el núcleo del desarrollo para asegurar entregas frecuentes de valor:

- Enfoque de "Equipo Completo" (Whole-Team Approach) → Todos los miembros colaboran en las tareas de prueba y son responsables de la calidad final.
- Desarrollo Guiado por Pruebas (TDD) → Escribir pruebas unitarias antes del código para guiar el diseño y prevenir errores.
- Desarrollo Guiado por Pruebas de Aceptación (ATDD) → Crear pruebas basadas en ejemplos de negocio antes de codificar cada historia de usuario.
- Integración Continua (CI) → Integrar código frecuentemente y ejecutar pruebas automatizadas para obtener feedback inmediato.
- Automatización de Pruebas de Regresión → Crear una red de seguridad automatizada que permita cambios constantes sin romper funcionalidades existentes.
- Testing Exploratorio → Pruebas manuales simultáneas al aprendizaje del sistema para encontrar fallos que la automatización no detecta.
- Uso de los Cuadrantes del Testing Ágil → Una matriz para planificar y asegurar que se cubren pruebas tecnológicas, de negocio, de apoyo al equipo y de crítica al producto.
- Pirámide de Automatización → Priorizar una base amplia de pruebas unitarias sobre una capa menor de pruebas de interfaz gráfica (GUI) para optimizar el retorno de inversión.
- Retrospectivas → Evaluar regularmente el proceso de prueba para identificar y eliminar obstáculos.

Los Cuadrantes del Testing Ágil:

Este modelo sirve como guía para asegurar que se cubran todos los tipos de pruebas necesarios:

- Cuadrante 1 (Tecnología - Apoyo al equipo) → Pruebas unitarias y de componentes, generalmente automatizadas, que ayudan a los programadores a diseñar mejor el código.
- Cuadrante 2 (Negocio - Apoyo al equipo) → Pruebas funcionales, ejemplos y pruebas de historias de usuario (ATDD) que definen los requerimientos de forma ejecutable
- Cuadrante 3 (Negocio - Crítica al producto) → Pruebas manuales como el testing exploratorio, pruebas de usabilidad y de aceptación de usuario (UAT) para evaluar si el producto satisface al cliente
- Cuadrante 4 (Tecnología - Crítica al producto) → Pruebas no funcionales como las de rendimiento, carga, seguridad y estabilidad, que requieren herramientas especializadas

La Estrategia de Automatización y la Pirámide

Dado que las iteraciones son rápidas (1 a 4 semanas), la automatización es vital para mantener una velocidad constante y proporcionar una red de seguridad contra regresiones. Se utiliza la Pirámide de Automatización para optimizar esfuerzos:



- Base → Gran volumen de pruebas unitarias rápidas y económicas
- Medio → Pruebas de aceptación a nivel de API o "detrás de GUI"
- Cima → Pocas pruebas a nivel de interfaz de usuario (GUI), ya que son más costosas y frágiles
- Nube → Testing exploratorio y manual que aporta la visión humana que la automatización no puede captar

Ciclo de Vida del Tester en una Iteración

- Planificación → Los testers ayudan a definir los criterios de aceptación y la "definición de hecho" (Definition of Done) para cada historia de usuario
- Ejecución → Mientras el desarrollador codifica, el tester prepara las pruebas. Ambos colaboran estrechamente: el programador escribe pruebas unitarias y el tester automatiza las de alto nivel o realiza pruebas exploratorias sobre el código disponible
- Cierre → Se realiza una demostración del incremento funcional al cliente para obtener feedback y se celebra una retrospectiva para identificar cómo mejorar el proceso de prueba en el siguiente ciclo.

Comparación entre Tradicional y Ágil

Característica	***Testing Tradicional (Cascada)***	***Testing Ágil***
Objetivo Principal	Encontrar defectos una vez que el código está terminado (enfoque destructivo).	Prevenir defectos mediante la colaboración constante y pruebas tempranas (enfoque preventivo).
Momento de Ejecución	Es una fase final y aislada que ocurre después de la codificación.	Es una actividad continua e integrada desde el inicio del desarrollo.
Responsabilidad	Recae exclusivamente en un equipo de QA independiente o "Policía de Calidad".	Enfoque de "Equipo Completo"; todos son responsables de la calidad (programadores, testers, clientes).
Requerimientos	Se definen de forma exhaustiva y se congelan al inicio del proyecto.	Son evolutivos y se definen "justo a tiempo" a través de Historias de Usuario.

Ciclo de Vida	Secuencial (Phased/Gated). El progreso es lineal y por etapas rígidas.	Iterativo e Incremental. Se desarrolla y prueba en ciclos cortos (1-4 semanas).
Retroalimentación	Latencia larga; los resultados se conocen meses después de definir los requerimientos.	Retroalimentación rápida y constante que guía el desarrollo día a día.
Documentación	Pesada y formal: Planes de prueba detallados, ERS y matrices de trazabilidad.	Ligera: Las pruebas automatizadas y criterios de aceptación sirven como documentación viva.
Automatización	A menudo se centra en la interfaz de usuario (GUI) y se hace al final.	Vital desde el inicio: Basada en la Pirámide de Automatización (muchas pruebas unitarias y de API).
Relación con Programadores	A veces adversaria ("el tester contra el programador").	Colaborativa; trabajan a la par (Pair Testing) para asegurar que se construye el producto correcto

Desarrollo Conducido por Testing (TDD)

El Desarrollo Guiado por Pruebas (TDD) es una metodología ágil donde la creación de pruebas unitarias procede a la escritura del código fuente. El proceso se fundamenta en el ciclo iterativo Red-Green-Refactor, el cual busca minimizar errores mediante la validación constante y la mejora de la estructura interna del software.

Desarrollo Tradicional vs TDD

Aspecto	*Tradicional*	*TDD*
Orden de las actividades y ciclo de vida	Sigue una secuencia lineal donde primero se diseña, luego se desarrolla (codifica) y finalmente se prueba. Esto suele asociarse al modelo en cascada, donde las fases son consecutivas y los errores graves pueden no detectarse hasta las etapas finales del proyecto	En este modelo, se escribe la prueba unitaria antes que el código de producción. El ciclo consiste en: 1) Escribir un test que falle, 2) Escribir el código mínimo para que pase, y 3) Refactorizar para limpiar el diseño
Enfoque del Diseño	Se basa en una planificación exhaustiva previa y diseños de arquitectura complejos antes de escribir una sola línea de código de negocio, lo que puede llevar a malgastar recursos en funcionalidades innecesarias	Promueve un diseño emergente u orgánico. La arquitectura se forja a base de iterar y refactorizar, permitiendo que las decisiones de diseño se tomen en el momento oportuno basándose en los requisitos actuales y no en especulaciones futuras

Retroalimentación y Gestión de Defectos	El control de calidad tiende a ser reactivo. La brecha entre una decisión y su retroalimentación puede ser de semanas o meses, lo que dificulta la corrección de errores	El aseguramiento de calidad es proactivo. Proporciona una retroalimentación continua y rápida (en segundos o minutos), lo que reduce drásticamente la densidad de defectos y transforma el miedo a los cambios en confianza
Documentación y Requisitos	Los requisitos suelen expresarse en prosa de lenguaje natural o documentos extensos que pueden dar pie a imprecisiones y malas interpretaciones	Los requisitos se transforman en ejemplos ejecutables (tests de aceptación). Estos tests sirven como la mejor documentación técnica posible, ya que son precisos, no ambiguos y la máquina puede validarlos automáticamente
Control de Alcance	Existe una tendencia a "diseñar para el mañana", lo que a menudo resulta en código complejo y sobrediseñado	Se rige por la regla de no escribir código que no sea necesario para que el test pase (principio YAGNI: You Ain't Gonna Need It). Esto asegura que solo se implementen las funciones justas que el cliente necesita

Leyes del TDD

1. No escribirás una sola línea de código de producción hasta que no hayas escrito primero un caso de prueba (test case) que falle.
2. No escribirás más de una prueba unitaria de lo necesario para que falle, y no compilar es fallar. El documento indica que normalmente con un test que falle es suficiente.
3. No escribirás más código de producción que el necesario para que la prueba pase

El Desarrollo de Software Ágil con TDD

El desarrollo de software ágil con TDD se basa en crear el sistema de manera iterativa y guiada por pruebas. Primero se define una historia de usuario y los escenarios que describen el comportamiento esperado del sistema. A partir de esos escenarios se crean pruebas de aceptación (ATDD) que inicialmente fallan, ya que la funcionalidad todavía no está implementada. Luego, utilizando TDD, se desarrollan pruebas unitarias pequeñas que también fallan al comienzo. Después se escribe el código mínimo necesario para hacer pasar las pruebas y finalmente se refactoriza el código para mejorarlo sin modificar su funcionamiento. Este proceso se repite continuamente hasta obtener un software funcional y alineado con las necesidades del usuario.

Por ende, en un entorno ágil con TDD:

- El trabajo se divide en iteraciones cortas (típicamente de 2 a 6 semanas).
- Se implementan historias de usuario prioritarias que deben terminarse en máximo una semana para entregar valor real al cliente rápidamente.

- El diseño es emergente u orgánico: la arquitectura no se define por completo al inicio, sino que se forja y evoluciona a base de iterar y refactorizar según las necesidades de los ejemplos actuales.

También cambia la forma de comunicarse con el cliente:

- Los requisitos escritos en prosa ambigua se reemplazan por ejemplos ejecutables o tests de aceptación.
- El equipo trabaja en una jerarquía horizontal donde el cliente y los analistas colaboran diariamente con los desarrolladores para definir estos criterios de aceptación antes de empezar a programar.
- Las pruebas se convierten en la mejor documentación técnica, ya que son precisas, no ambiguas y siempre están actualizadas

Beneficios del TDD:

- Código Robusto
    - Código más predecible y limpio, que permite saber cuándo el código está terminado
    - Código más tolerable al cambio, al tener escritas las pruebas es más fácil de entender
    - Código más seguro
    - Aumenta la posibilidad de reducir la duplicación de código (si el test pasa el código ya existe
- Código más barato de mantener al ser más entendible
- Con la práctica acelera la velocidad de desarrollo y la posibilidad de despliegue continuo
- Asegura cobertura de prueba, lo que conlleva a código de calidad
- Obliga a pensar en cómo usar el componente, por lo tanto, el diseño de las API
- Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento.

Pasos del TDD

El ciclo Red-Green-Refactor

El núcleo técnico del TDD es un ciclo de retroalimentación de segundos o minutos que consta de tres pasos:

- Rojo (Red) → Escribir una pequeña prueba automatizada para una funcionalidad que aún no existe y verla fallar. Esto transforma el problema de "crear un sistema complejo" a "hacer que esta prueba pase".
    - En esta etapa, se diseña la interfaz o API que nos gustaría tener, tratando la prueba como una especificación del comportamiento deseado
    - Si la prueba ni siquiera compila, esto ya se considera una falla necesaria para avanzar
- Verde (Green) → Escribir el código mínimo indispensable para que la prueba pase, incluso si la solución no es perfecta en ese momento. El objetivo absoluto en este punto es conseguir la "barra verde" en segundos o minutos.

    - Kent Beck sugiere que en este paso se pueden "cometer pecados" (como duplicar datos o usar constantes fijas) con tal de llegar rápido al estado funciona
- Refactorizar (Refactor) → Eliminar la duplicidad y mejorar el diseño del código recién escrito sin alterar su comportamiento, asegurando que todas las pruebas sigan pasando
    - Se aplican principios de diseño como S.O.L.I.D. para asegurar que el código sea mantenible y limpio
    - Es vital volver a ejecutar todas las pruebas tras cada pequeño cambio para confirmar que todo sigue funcionando correctamente

Pasos

1. Escribir un test.
2. Hacer que el test compile.
3. Hacer que el test falle (Red).
4. Realizar un cambio mínimo para que pase.
5. Correr el test para que pase (Green).
6. Quitar duplicaciones (Refactor)

Patrones de TDD

Patrones red bar

Los patrones de barra roja se enfocan en decidir qué pruebas escribir, dónde escribirlas y cuándo dejar de hacerlo para mantener el ritmo de desarrollo.

- ***One Step Test*** →*Escribe un solo test que verifique una pieza mínima de funcionalidad.*
    - Consiste en elegir de la lista de tareas un test que el desarrollador esté seguro de poder implementar y que, al mismo tiempo, le enseñe algo nuevo. Cada test debe representar un paso pequeño y manejable hacia el objetivo final; si no hay ninguno que parezca un paso sencillo, se deben añadir tests más pequeños que representen progreso
- ***Starter Test*** → *Primer test que te ayuda a explorar la API o comportamiento deseado.*
    - Para comenzar una nueva operación, se debe probar una variante que no haga nada. Este test ayuda a responder la pregunta estratégica de "¿a dónde pertenece esta operación?" sin tener que preocuparse todavía por los inputs o resultados correctos de un caso real, acortando así el ciclo de retroalimentación
- ***Explanation test*** → *Escribe comentarios o preguntas para aclarar qué esperas que haga el código.*
    - Se utiliza para difundir el uso de las pruebas y clarificar requisitos dentro de un equipo. En lugar de usar descripciones verbales ambiguas, se piden y dan explicaciones en forma de casos de prueba concretos (ej. "¿si tengo este valor y aquel otro, el resultado es 76?") para asegurar que todos entienden lo mismo.

- ***Learning test*** → *Antes de utilizar una nueva funcionalidad de un software o biblioteca externa, escribe una pequeña prueba que verifique que la API funciona como esperas. Esto asegura que comprendes la herramienta antes de integrarla en tu código de producción*
- ***Another Test*** → *Añade un nuevo test para cubrir un caso adicional o una variante del comportamiento.*
    - Es una técnica de gestión del foco. Cuando durante una discusión técnica surge una idea tangencial o una duda, en lugar de desviarse del tema principal, se añade esa idea a la lista de tests pendientes y se retoma el trabajo actual, evitando así procrastinar o perder el hilo del desarrollo
- Regression Test → *Agrega un test que reproduce un bug descubierto, para evitar regresiones futuras*.
    - Es el primer paso que se debe dar cuando se reporta un defecto. Se debe escribir el test más pequeño posible que falle reproduciendo el bug; una vez que el test pase, el error se considera reparado y el test servirá para evitar que el problema vuelva a aparecer en el futuro

Patrones green bar

Los patrones de barra verde tienen como objetivo principal lograr que las pruebas pasen lo más rápido posible, incluso si la solución inicial no es perfecta. Una vez que el sistema está en "verde", se tiene la base necesaria para refactorizar con confianza.

- ***Fake It*** → Es la forma más rápida de llegar al verde tras un fallo. Consiste en devolver un valor constante o una solución "falsa" directamente en el código de producción.
    - Una vez que la prueba pasa, se transforma gradualmente esa constante en una expresión real mediante la eliminación de la duplicidad de datos entre la prueba y el código.
- ***Triangulación*** → Es la estrategia más conservadora para abstraer código. Consiste en no generalizar la solución hasta que se tengan dos o más ejemplos (tests) que la demanden.
    - Se recomienda utilizarlo cuando no se tiene claro cómo eliminar la duplicación de forma correcta o cuando el diseño no fluye de manera obvia.
    - Si un test pasa con un valor fijo (Fake It), se añade un segundo test con datos diferentes que fuerce a escribir la lógica general para que ambos puedan pasar simultáneamente.
- ***Obvious Implementation*** → Cuando la solución es trivial y obvia, puedes implementarla directamente sin "Fake It".
    - Riesgo → Si al intentar una implementación obvia aparece una "barra roja" inesperada, se debe "reducir la marcha" y volver a los pasos más pequeños de los otros patrones.
- ***One to many*** → Patrón para resolver test para colecciones de elementos
    - Primero se escribe el test y la implementación para un solo elemento. Una vez funciona, se modifica el código para que acepte una colección, se migra la

lógica para que procese dicha colección y finalmente se eliminan los parámetros individuales.
    - Permite aislar los cambios y realizar una migración de datos controlada sin romper las pruebas existentes.

Patrones TDD

Los patrones de desarrollo guiado por pruebas (TDD) proporcionan estrategias fundamentales para organizar las pruebas y guiar el diseño del software

- ***Test*** → Representa un caso de prueba individual que se implementa como un método cuyo nombre comienza con la palabra "test" por convención

Nta: Es para una rápida identificación para las herramienta y para que los que leen el test entiendan mejor para que es.

- Isolated test → Mantener los test totalmente independientes

Nta: Esto hace que se identifiquen problemas rápidamente y fomenta el diseño con alta cohesión y bajo acoplamiento.

- ***Test List*** → Consiste en escribir una lista de todas las pruebas que se planean realizar antes de empezar a programar. Esta lista ayuda a gestionar el estrés, permite mantener el enfoque en la tarea actual y sirve como una hoja de ruta para saber qué falta por implementar y refactorizar
- ***Test first*** → Escribir la prueba antes de hacer el código. Utilizamos la prueba como una herramienta de diseño y control de alcance, reduciendo la incertidumbre en el desarrollo.
- ***Assert test*** → primero escribir la confirmación y luego la prueba

Nta: Lo más importante sería la respuesta correcta/que es lo que tiene que ocurrir para que pase y después concentrarse en los detalles.

- ***Test data*** → Recomienda usar datos que sean fáciles de leer y seguir para la audiencia de la prueba. Se debe evitar el uso de constantes iguales para propósitos diferentes y preferir valores que tengan un significado conceptual claro, facilitando la distinción de los argumentos en la implementación
- ***Evident data*** → Consiste en incluir los resultados esperados y reales de forma que la relación entre ellos sea obvia o evidente.
    - En lugar de poner un valor final precalculado, se prefiere escribir la operación o el razonamiento que lleva a ese resultado.

Otros Patrones

- ***Patrones de testing*** → Son estrategias para solucionar dificultades técnicas al escribir las pruebas, como el manejo de recursos complejos o la verificación de comportamientos difíciles de aislar.

Nta: Me hacen la vida más fácil cuando la prueba es muy grande o depende de cosas externas complicadas de usar.

- ***Patrones de diseño*** → Son estructuras arquitectónicas que se utilizan para organizar el código de producción durante el paso de refactorización. Ayudan a que el diseño del software emerja de manera orgánica, asegurando que la solución final sea flexible, escalable y libre de duplicación (código limpio)
- ***xUnit Patterns*** → Definen la forma estándar de estructurar y automatizar las pruebas utilizando marcos de trabajo de testing (como JUnit, NUnit o PyUnit). Se enfocan en

cómo organizar los métodos de prueba, preparar los datos de contexto necesarios y verificar los resultados de manera que las herramientas puedan identificar y ejecutar todo el proceso de forma automática

Refactoring

El refactoring (refactorización) es una técnica disciplinada que consiste en modificar la estructura interna del código sin alterar su comportamiento externo o funcionalidad observable. Su objetivo principal es transformar un diseño de software que simplemente "funciona" en uno que sea limpio, legible y fácil de mantener.

Los cambios deberían:

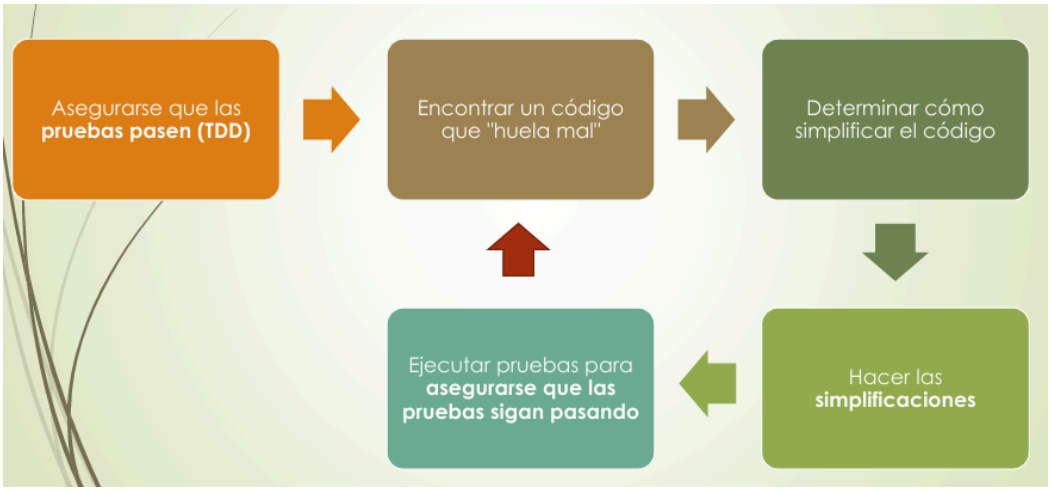
- Eliminar la duplicación → Busca borrar cualquier lógica o dato repetido que haya surgido para hacer pasar el test.
- Simplificar la complejidad → Reduce el desorden y la complejidad innecesaria, haciendo que el código sea más simple de entender.
- Mejorar la legibilidad → Transforma el código para que sirva como documentación técnica, facilitando que otros desarrolladores comprendan la intención original.
- Facilitar el cambio → Un código bien refactorizado es más tolerable al cambio y permite que nuevas funcionalidades se integren con menor riesgo de introducir errores.
- No cambien el comportamiento observable (todas las pruebas aún pasan)
- Flexibiliza el código

¿Por qué es necesario hacer refactoring?

- ***Evitar el "deterioro del diseño" y Limpiar desorden en el código*** → Durante la fase de "Verde" en TDD, el objetivo es hacer que el test pase lo más rápido posible, incluso "cometiendo pecados" o escribiendo código "sucio". El refactoring es el paso disciplinado para eliminar la complejidad innecesaria y mejorar la estructura interna, evitando que el sistema se vuelva inmanejable o frágil ante cambios
- ***Simplificar el código*** → Al remover lógica repetida y simplificar las estructuras (como reemplazar condicionales con polimorfismo), el código se vuelve más flexible y fácil de cambiar
- ***Incrementar la legibilidad y la comprensibilidad*** → El código bien refactorizado funciona como la mejor documentación técnica posible. Al usar nombres descriptivos y estructuras claras, se facilita que otros desarrolladores (o uno mismo en el futuro) entiendan la intención original del autor sin necesidad de comentarios extensos
- ***Encontrar errores y Reducir el tiempo de depuración*** → El diseño de tests y el refactoring ayudan a descubrir y eliminar problemas en todas las etapas del desarrollo. Al mantener un código limpio que funciona, se eliminan los largos rastros de errores y se reduce drásticamente la necesidad de usar el depurador, ya que los fallos se vuelven evidentes de inmediato
- ***Incorporar el aprendizaje que hacemos sobre la aplicación*** → A medida que se implementa una funcionalidad, el desarrollador comprende mejor el problema; el refactoring permite plasmar ese nuevo conocimiento en el diseño de forma orgánica
- ***Rehacer las cosas es fundamental en todo proceso creativo***

Implementación

1. ***Asegurarse de que las pruebas pasen*** → Antes de tocar el código, el sistema debe estar en estado "Verde" (todas las pruebas actuales deben ser exitosas).
2. ***Encontrar un código que "huela mal" (code smell)*** → Se debe identificar una estructura problemática, como lógica duplicada, métodos demasiado largos o dependencias innecesarias.
3. ***Determinar cómo simplificar el código*** → Se decide qué transformación aplicar para mejorar el diseño sin alterar el comportamiento funcional.
4. ***Hacer las simplificaciones en pasos atómicos*** → Se aplica un cambio pequeño (por ejemplo, renombrar una variable o extraer un método).
5. ***Ejecutar las pruebas inmediatamente*** → Tras cada pequeño cambio, se corren los tests para verificar que no se ha roto nada. Si los tests fallan, se deshace el cambio y se vuelve a intentar en un paso más pequeño



Resumen del TDD

